

From TS-SUF-2 to TS-SUF-4: Practical Security Enhancements for FROST2 Threshold Signatures

Will Wang, Syh-Yuan Tan, Ryan Chow, Chanson Chan, and Martin Zhao

Solv Protocol, Singapore

{will,sytan,ryan,chanson,martin}@solv.finance

Abstract. Threshold signature schemes play a vital role in securing digital assets within blockchain and distributed systems. FROST2 stands out as a practical threshold Schnorr signature scheme, noted for its efficiency and compatibility with standard verification processes. However, under the one-more discrete logarithm assumption, with static corruption and centralized key generation settings, FROST2 has been shown by Bellare et al. (in CRYPTO 2022) to achieve only TS-SUF-2 security, which is a consequence of its vulnerability to TS-UF-3 attacks.

In this paper, we address this security limitation by presenting an enhanced variant of FROST2, namely, FROST2+ which achieves the TS-SUF-4 security level under the same computational assumptions as the original FROST2. FROST2+ strengthens FROST2 by integrating additional pre-processing token verifications that help mitigate TS-UF-3 and TS-UF-4 vulnerabilities while maintaining practical efficiency. We show that FROST2+ can achieve TS-SUF-4 security not only under the same conditions as the original FROST2 analysis, but also when initialized with a distributed key generation protocol such as PedPoP. Our benchmark using ZCash’s FROST library shows that the performance of FROST2+ is comparable to FROST2 and about 64 – 79% faster than FROST when precomputation is enabled.

Keywords: Schnorr · Threshold Signature · FROST · TS-SUF · Provable Security

1 Introduction

The Schnorr signature scheme [28] has emerged as a cornerstone for blockchain applications due to its efficient signing and verification processes, compact signatures, and support for aggregation. In this context, Schnorr-based threshold signatures are particularly critical for threshold signing in blockchain systems [24,20,1,27], with FROST [22] establishing itself as a benchmark and currently undergoing IETF standardization [12]. Furthermore, NIST’s recent call for contributions on threshold multi-party schemes [9] has intensified research interest in FROST.

Research related to FROST can be categorized into two main areas: the development of new schemes that offer enhanced efficiency [15,8,6,23], and improvements in security [26,11,2,10,3,25,4]. The latter category can be further divided

into specific areas of security analysis focused on distributed key generation [11,15,6] and adaptive security [16,14,10,18,2,25,17,4]. Several works [8,6,13,14,18] delve deeper into the security notions relevant to threshold signature schemes.

The TS-(S)UF- i security framework [8,6] represents a recent effort to rigorously formalize the security of threshold signature schemes. This framework groups a message M along with its auxiliary signing information as a leader request (lr) and classifies unforgeability levels according to adversarial capabilities, ranging from the weakest (TS-UF-0, which resists less capable attackers) to the strongest (TS-UF-4, which resists more capable attackers). While the framework typically describes forgeries in terms of triviality, let $c < t$ denotes the number of corrupted signers, we outline the attackers' abilities at each level for clarity:

1. TS-UF-0: The threshold scheme can withstand an attacker who cannot obtain any partial signatures from honest signers for the target message M .
2. TS-UF-1: The threshold scheme can withstand an attacker at TS-UF-0 who also queries at most $t - c - 1$ partial signatures on M from honest signers.
3. TS-UF-2: The threshold scheme can withstand an attacker at TS-UF-1 who also queries leader requests (lr) on M and obtain at most $t - c - 1$ honest partial signatures per lr .
4. TS-UF-3: The threshold scheme can resist an attacker at TS-UF-2 who also corrupts pre-processing tokens (pp_i) for honest signers $id_i \in lr.S$ in the signing set, but not for all non-responding honest signers in the same set.
5. TS-UF-4: The threshold scheme can resist an attacker at TS-UF-3 who corrupts pre-processing tokens (pp_i) for all honest signers $id_i \in lr.S$ in the signing set, though it cannot obtain partial signatures from every honest signer in $lr.S$.

where the descriptions of TS-UF-3 and TS-UF-4 refer to the adversary capabilities in echo schemes [6,8,13] like FROST. In such schemes, an honest signer id_i only produces a partial signature if the leader request (lr) contains id_i 's honestly generated pre-processing token $pp_i \in lr.PP(id_i)$. The framework also introduces the concepts of strong unforgeability (TS-SUF- i), which ties a request lr to each unique signature, beyond just the message M .

Moreover, the framework provides valuable insights into the relationships among security levels. For instance, it demonstrates that achieving TS-UF-0 security implies TS-UF-1 security, risking a security loss of $\binom{n}{t-1}$ for number of signers n and threshold t . Additionally, it shows that combining a threshold scheme achieving TS-UF-3 security with a standard digital signature scheme with strong existential unforgeability under chosen-message attacks (SUF-CMA) results in a threshold scheme with TS-UF-4 security.

Within this security framework, both FROST and FROST2 achieve TS-UF-0 (and thereby TS-UF-1) security in the distributed key generation (DKG) settings [22,15,6], which represents the baseline security level within the TS-UF- i hierarchy. Under the centralized key generation (CKG) setting, FROST achieves TS-SUF-3 security and can be further improved to attain the highest TS-SUF-4 security level by incorporating an additional digital signature scheme for each signer

[8,6,13]. However, this approach introduces additional computational overhead and implementation complexity. More critically for our work, FROST2 [15,8,6] has been shown to achieve only TS-SUF-2 security and is not qualified for this security upgrade.

The limitation of FROST2 to TS-SUF-2 security arises from a fundamental vulnerability to TS-UF-3 attacks, as demonstrated by Bellare et al. [8,6]. This attack is reminiscent of the rogue public key attack in multi-signature schemes, exploiting a weakness in FROST2’s pre-processing token mechanism to forge a Schnorr signature. Consequently, this enables a forgery under an unauthorized signer set without needing to know the partial signatures of all honest signers, thereby compromising TS-UF-3 security (see Appendix A.2).

Enhancements to FROST2 have been proposed, but none have achieved TS-SUF-3 security or higher. For instance, ROAST [26] introduces token aggregation and signature share verification, improving robustness while acknowledging security reduction to the original FROST2. Its successor, Olaf [11], combines ROAST with a simplified DKG protocol (SimplPedPoP) from FROST’s PedPoP, proving its TS-SUF-2 security. Recent theoretical results [14,17] indicate that FROST, FROST2, and FROST3 (or ROAST) are either unforgeable with full adaptive security in the algebraic group model (AGM), under the assumptions of Algebraic One-More Discrete Logarithm (AOMDL) and Low-Dimensional Vector Representation (LDVR); or adaptively secure with a maximum of $t/2$ corruptions, assuming AOMDL in the random oracle model (ROM).

FROST-Mask [10], which can be viewed as an extension of FROST2 with masking mechanisms applied to the pre-processing tokens and partial signatures, has been proven unforgeable with adaptive security under the ROM. Conversely, ms-FROST [2] and FaFROST [4] apply masking solely to the partial signature, proving to achieve TS-UF-4 and adp-TS-UF, respectively, with adaptive security under the weaker AGM.

The security notions TS-SUF-3 and TS-SUF-4 are important in practice. The TS-SUF-3 security protects against premature leakage of a pre-processing token, and the TS-SUF-4 security prevents an honest signer from being wrongfully framed for a forgery. In this work, we investigate whether FROST2, which improves efficiency over FROST and reduces communication rounds compared to several later three-round variants [15,3,25], can achieve the same TS-SUF-3 and TS-SUF-4 security levels as FROST in the ROM.

1.1 Our Contributions

We proposed an improved variant of FROST2 that achieve TS-SUF-4 security, the highest tier in Bellare et al.’s hierarchical framework, while maintaining the protocol’s efficiency and its two-round structure.

The proposed FROST2+ addresses the TS-UF-3 vulnerability of FROST2 by introducing a lightweight token authentication mechanism for pre-processing tokens. In contrast to Bellare et al.’s ATs[TS, DS] transformation [8,6], which requires each signer to generate and manage an additional digital signature key-pair, our approach authenticates the tokens using only the signing key share

and the secret exponents already presented in the pre-processing tokens (R, S) . We establish the TS-SUF-4 security of FROST2+ under the one-more discrete logarithm (OMDL) assumption within the ROM, in the static corruption and centralized key generation (CKG) settings.

Subsequently, we show that our proposed schemes maintain their security levels when transitioning from the CKG setting to a distributed key generation (DKG) setting, specifically using PedPoP DKG as an instantiation. We also explain how to speed up the partial signing algorithm by pre-verifying the pre-processing tokens, which is not bind to the signing message. Moreover, similar to FROST’s offline pre-processing, our token verification mechanism facilitates per-signer batching across multiple signing sessions. This design choice preserves the efficiency of FROST-style pre-processing, making our schemes particularly advantageous for high-throughput, leaderless environments where offline token preparation is critical.

Finally, we show that the proposed FROST2+, while offering stronger security, achieves better efficiency than FROST and matches the performance of FROST2, as detailed in our performance evaluation (Section 5).

Organization. This paper is structured as follows. In Section 2, we present the necessary mathematical tools. We then outline the proposed FROST2+ scheme in Section 3, followed by the security analysis in Section 4. Next, we present the performance evaluation in Section 5 and conclude the paper in Section 6.

2 Preliminaries

We briefly recall the required mathematical tools in this section.

2.1 One-More Discrete Logarithm (OMDL) Assumption

Definition 1. Let $\mathbb{G}$ be a cyclic group of prime order p with generator g , and let CHALL and DLOG be oracles such that:

- CHALL() returns a random challenge $A = g^a$ for $a \xleftarrow{\$} \mathbb{Z}_p^*$
- DLOG(B) returns $b \in \mathbb{Z}_p^*$ such that $B = g^b$ for any $B \in \mathbb{G}$

The OMDL assumption states that for any probabilistic polynomial-time adversary $\mathcal{C}$ making at most $q - 1$ queries to DLOG, the advantage

$$\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C}) = \Pr \left[\begin{array}{l} (A_1, \dots, A_q) \leftarrow \text{CHALL}(), \\ (a_1, \dots, a_q) \leftarrow \mathcal{C}^{\text{CHALL}, \text{DLOG}}(g) \\ \forall i : g^{a_i} = A_i \wedge \mathcal{C} \text{ never queried DLOG}(A_i) \end{array} \right]$$

is negligible.

2.2 Lagrange Coefficient

Let $f(\cdot)$ be a random $(t - 1)$ -degree polynomial such that $f(x_i) = s_i$. The polynomial $f(\cdot)$ can be reconstructed by having the knowledge of t -pair values of (x_i, s_i) :

$$f(\cdot) = \sum_{x_i \in \mathcal{S}} s_i \Delta_{x_i, \mathcal{S}}(\cdot)$$

where $\mathcal{S} = \{x_1, x_2, \dots, x_t\}$.

Definition 2. The Lagrange coefficient $\Delta_{x_i, \mathcal{S}}$ is defined as:

$$\Delta_{x_i, \mathcal{S}}(Z) = \prod_{x_j \in \mathcal{S}, j \neq i} \frac{Z - x_j}{x_i - x_j}$$

where x_i and x_j are elements in the set $\mathcal{S}$.

2.3 Threshold Signature Scheme

To synchronize with Bellare et al.'s TS-(S)UF- i security framework [8,6], we adopt their scheme definition while making some adjustments, which includes the addition of a **Setup** algorithm and modifications to certain inputs for each algorithm. We define a non-interactive threshold signature scheme as $\text{TS} = \{\text{Setup}, \text{Kg}, \text{SPP}, \text{LPP}, \text{LR}, \text{PS}, \text{Agg}, \text{Vf}, \text{SVf}\}$, characterized as follows:

$\text{Setup}(1^\lambda) \rightarrow \text{par}$: The setup algorithm takes in a security parameter 1^λ and produces public parameters par , which are implicitly provided to all other algorithms.

$\text{Kg}(\text{par}, t, \text{ns}) \rightarrow \{\text{st}_i = (\tilde{X}, \text{aux}, \text{sk})\}_{i=1}^{\text{ns}}$: The key generation algorithm takes in threshold t and number of signers ns such that $t \leq \text{ns}$, initializes the state st_i for every signer $\text{id}_i \in \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}$ with joint public key $\tilde{X}$, public auxiliary information aux , and a signing key share sk .

$\text{SPP}(\text{st}_i) \rightarrow (pp, \text{st}_i)$: The signing pre-processing algorithm takes as input the state st_i of signer id_i . It outputs a pre-processing token pp and updates the signer's state.

$\text{LPP}(\text{id}_i, pp, \text{st}_0) \rightarrow \text{st}_0$: The leader pre-processing algorithm takes the pre-processing token pp from signer id_i and the leader's state st_0 , updating st_0 to incorporate the token.

$\text{LR}(M, \mathcal{S}, \text{st}_0) \rightarrow (lr, \text{st}_0)$: This leader signing-request algorithm generates a leader request lr based on a message M , a signing set $\mathcal{S}$ with at least t signers, and the leader's state st_0 .

$PS(lr, st_i) \rightarrow (psig_i, st_i)$: The signer partial-signing algorithm takes the leader request lr , the state st_i of a signer id_i , generating a partial signature $psig_i$ and updating state st_i .

$Agg(lr, \{psig_i\}_{id_i \in lr.S}, st_0) \rightarrow (\sigma, st_0)$: The leader partial-signature aggregation algorithm aggregates the partial signatures received from the signing set $\mathcal{S}$ into a final signature σ for the message M while updating the leader's state st_0 .

$Vf(\tilde{X}, M, \sigma) \rightarrow 0$ or 1 : The verification algorithm checks the validity of the signature σ for the message M using the public key $\tilde{X}$, returning 1 if valid and 0 otherwise.

$SVf(\tilde{X}, lr, \sigma) \rightarrow 0$ or 1 : The strong verification algorithm checks the validity of the signature σ for the leader request lr using the public key $\tilde{X}$, returning 1 if valid and 0 otherwise.

2.4 Security Model

Table 1. Trivial forgery predicates for TS-(S)UF- i [8,6] with Cremers et al.'s recommendation [13].

Predicate	Definition
$tf_0(M)$	$S_1(M) = \emptyset$
$tf_1(M)$	$ S_1(M) \geq t - CS $
$tf_2(lr)$	$ S_2(lr) \geq t - CS $
$tf_3(lr)$	$tf_2(lr) \wedge S_2(lr) = S_3(lr)$
$tf_4(lr)$	$tf_2(lr) \wedge S_2(lr) = S_3(lr) = S_4(lr)$
$tsf_2(lr, \tilde{X}, \sigma)$	$tf_2(lr) \wedge SVf(\tilde{X}, lr, \sigma)$
$tsf_3(lr, \tilde{X}, \sigma)$	$tf_3(lr) \wedge SVf(\tilde{X}, lr, \sigma)$
$tsf_4(lr, \tilde{X}, \sigma)$	$tf_4(lr) \wedge SVf(\tilde{X}, lr, \sigma)$

To address a shortcoming in the original hierarchy of Bellare et al.'s hierarchy framework [6], where TS-(S)UF-4 does not imply TS-(S)UF-3, and vice versa, due to incomparable trivial forgery conditions, we adopt the strengthened trivial forgery condition in Table 1 for tf_4 as recommended by Cremers et al. [13]. Specifically, this strengthens tf_4 from requiring $S_2(lr) = S_4(lr)$ to $S_2(lr) = S_3(lr) = S_4(lr)$, ensuring that all honest signers in $lr.S$ responded with honestly generated pre-processing tokens. The modification is purely definitional such that for echo schemes satisfying the standard pre-processing token check, it does not change the set of trivial forgeries. In particular, this holds for the ATS[TS,DS] transformation [8,6] whose partial signing only accepts signed pre-processing tokens, so the original and the strengthened tf_4 coincide and the existing TS-SUF-4 security proof for ATS[TS,DS] remains valid.

Games $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$ ($i = 0, 1, 2, 3, 4$) and $\mathbf{G}_{\text{TS}}^{\text{ts-suf-}i}$ ($i = 2, 3, 4$)

INIT(CS):

- 1: **Require:** $CS \subseteq \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}$ and $|CS| < t$ ▷ Set of corrupted signers
- 2: $h \xleftarrow{\$} \text{TS.HF}; (\tilde{X}, \text{aux}, \text{sk}_1, \dots, \text{sk}_{\text{ns}}) \xleftarrow{\$} \text{Kg}$ ▷ Set of hash functions TS.HF
- 3: $HS \leftarrow \{\text{id}_1, \dots, \text{id}_{\text{ns}}\} \setminus CS$ ▷ Set of honest signers
- 4: **for** $\text{id}_i \in HS$ **do**
- 5: $\text{st}_i.\text{sk} \leftarrow x_i; \text{st}_i.\tilde{X} \leftarrow \tilde{X}; \text{st}_i.\text{aux} \leftarrow \text{aux}$
- 6: **end for**
- 7: **return** $\tilde{X}, \text{aux}, \{x_i\}_{\text{id}_i \in CS}$

PPO(id_i):

- 8: **Require:** $\text{id}_i \in HS$
- 9: $(pp, \text{st}_i) \xleftarrow{\$} \text{SPP}(\text{st}_i); PPL \leftarrow PPL \cup \{(\text{id}_i, pp)\}$
- 10: **return** pp

PSIGNO(id_i, lr):

- 11: **Require:** $lr.S \subseteq \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}, M = lr.\text{msg} \in \{0, 1\}^*$ and $\text{id}_i \in HS$
- 12: $(\text{psig}, \text{st}_i) \xleftarrow{\$} \text{PS}(lr, \text{st}_i); L \leftarrow L \cup \{lr\}$
- 13: **if** $\text{psig} \neq \perp$ **then**
- 14: $S_1(M) \leftarrow S_1(M) \cup \{\text{id}_i\}; S_2(lr) \leftarrow S_2(lr) \cup \{\text{id}_i\}$
- 15: **end if**
- 16: **return** psig

RO[h](x):

- 17: **return** $h(x)$

FIN(M, σ):

- 18: **for all** $lr \in L$ **do**
- 19: $S_3(lr) \leftarrow \{\text{id}_i \in HS \cap lr.S : lr.\text{PP}(\text{id}_i) \in PPL\}; S_4(lr) \leftarrow HS \cap lr.S$
- 20: **end for**
- 21: **if** $\neg \forall f(\tilde{X}, M, \sigma)$ **then**
- 22: **return** false;
- 23: **end if**
- 24: **if** Game $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$ for $i = 0, 1$ **then**
- 25: **return** $\neg \text{tf}_i(M)$
- 26: **end if**
- 27: **if** Game $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$ for $i = 2, 3, 4$ **then**
- 28: **return** $\neg \exists lr(lr.\text{msg} = M \wedge \text{tf}_i(lr))$
- 29: **end if**
- 30: **if** Game $\mathbf{G}_{\text{TS}}^{\text{ts-suf-}i}$ **then**
- 31: **return** $\neg \exists lr(lr.\text{msg} = M \wedge \text{tsf}_i(lr, \tilde{X}, \sigma))$
- 32: **end if**

Fig. 1. Security games defining TS-(S)UF- i [6,8,13].

This strengthening approach, corresponds to Cremers et al.'s SiGu-4:aLRhPP level, restores linear downward implications across the hierarchy (e.g., TS-(S)UF-

4 implies TS-(S)UF-3) without weakening the lower levels as done in the revised version [8]. We prefer this strengthened TS-(S)UF- i model as shown in Figure 1 over Cremers et al.’s full framework [13] because it provides a standard comparison to FROST and its related variants.

Definition 3. Let $X \in \{\text{suf}, \text{uf}\}$ and $i \in \{0, 1, 2, 3, 4\}$. A TS scheme is secure if there is no polynomial time algorithm $\mathcal{F}$ that can win the game $\mathbf{G}_{\text{TS}}^{\text{ts-X-}i}$ in time $T(\mathcal{F})$ with non-negligible advantage:

$$\text{Adv}_{\text{TS}}^{\text{ts-X-}i}(\mathcal{F}) = \Pr[\mathbf{G}_{\text{TS}}^{\text{ts-X-}i}(\mathcal{F}) = \text{true}].$$

2.5 Generalized Forking Lemma Extension

This lemma is proposed by Bellare et al. [8,6] in proving the TS-SUF-2 security of FROST2. It extends the generalized Forking Lemma [7] where if $\mathcal{A}$ can only output index I within a given set $SS \subseteq [1..q]$, then the final bound on $\text{acc}(\text{Fork}^{\mathcal{A}})$ depends only on $|S|$ instead of q . This reduces the security loss from $1/q$ to $1/|SS|$.

Lemma 1. Let $q \geq 1$ be an integer, $SS \subseteq [1..q]$ be a set, and $\mathbb{H}$ be a set. Let $\mathcal{A}$ be a randomized algorithm that on input $x, h_1, \dots, h_q$ outputs a pair $(i_{\text{sig}}, \text{Out})$, where $i_{\text{sig}} \in \{\perp\} \cup SS$ and Out is a side output. Let GrGen be a randomized algorithm that generates x . The accepting probability of $\mathcal{A}$ is defined as $\text{acc}(\mathcal{A}) = \Pr_{x \leftarrow \text{GrGen}, h_1, \dots, h_q \leftarrow \mathbb{H}}[(i_{\text{sig}}, \text{Out}) \leftarrow \mathcal{A}(x, h_1, \dots, h_q) : i_{\text{sig}} \neq \perp]$.

The accepting probability of $\text{Fork}^{\mathcal{A}}$ is defined as

$$\text{acc}(\text{Fork}^{\mathcal{A}}) = \Pr_{x \leftarrow \text{GrGen}}[\alpha \leftarrow \text{Fork}^{\mathcal{A}}(x) : \alpha \neq \perp].$$

Then $\text{acc}(\text{Fork}^{\mathcal{A}}) \geq \text{acc}(\mathcal{A})^2/|SS|$.

2.6 Generalized Multi-Instance Multi-Forking Lemma

We extend the generalized multi-instance forking lemma of Bagherzandi et al. [5] to allow multiple forks per instance. The statement is average-case over parameters generated by GrGen , in the same style as the original lemma. In essence, we modify the success count of each local instance from 1 to n , analogous to Genaro et al.’s multi-reset lemma [19] built upon Bellare and Neven’s generalized reset lemma [7].

Let $\mathbb{H}$ be a finite set and let q, N, n be positive integers. Consider an algorithm $\mathcal{A}$ that, on input par , internal coins ρ , and a random-oracle tape $h = (h_1, \dots, h_q) \in \mathbb{H}^q$, outputs an index set I and side outputs $(\{\text{out}_i\}_{i \in I}, \theta)$. If $\mathcal{A}$ fails, it outputs $(I, \{\text{out}_i\}_{i \in I}, \theta)$ with $I = \emptyset$. Otherwise we assume $|I| = N$, treating I as a multiset and allowing repetitions, and we assume that if $I = \{i_1, \dots, i_N\}$ then $i_1 \leq \dots \leq i_N$. Note that we adopt the simplified output notations [11] that has an extra θ , which can be viewed as a suffix in every out_i .

Let $\Omega = \{(\rho, h_1, \dots, h_q) \mid \rho \in \{0, 1\}^*, h_j \in \mathbb{H}\}$ be the set of all possible random tapes. For a fixed par , let $S(\text{par}) = \{f \in \Omega \mid \mathcal{A}(\text{par}, f) \text{ succeeds}\}$. Define $\varepsilon(\text{par}) = \Pr_{f \leftarrow \Omega}[\mathcal{A}(\text{par}, f) \text{ succeeds}]$ and let $\hat{\varepsilon} = \Pr_{\text{par} \leftarrow \text{GrGen}(1^\lambda), f \leftarrow \Omega}[\mathcal{A}(\text{par}, f) \text{ succeeds}]$.

Let $\mathcal{O} = \{(h_{i_m}, \text{out}_{i_m})\}_{m=1}^N$ be the side outputs from the initial execution, and let $\mathcal{O}' = \emptyset$ initially. The forking algorithm $\text{MIMFork}_{N,n}^{\mathcal{A}}(\text{par})$ performs N local rewinding processes, one for each occurrence i_m in the ordered multiset I . For each target occurrence i_m , it resamples the random-oracle suffix starting at i_m until it collects n additional side outputs whose random-oracle responses at i_m are pairwise distinct and also distinct from the original response h_{i_m} .

Algorithm 1 Generalized Multi-Instance Multi-Forking Algorithm
 $\text{MIMFork}_{N,n}^{\mathcal{A}}(\text{par})$

```

1: Pick internal coins  $\rho$  for  $\mathcal{A}$  at random
2:  $(h_1, \dots, h_q) \xleftarrow{\$} \mathbb{H}^q$ 
3:  $(I, \{\text{out}_i\}_{i \in I}, \theta) \leftarrow \mathcal{A}(\text{par}, (h_1, \dots, h_q); \rho)$ 
4: if  $I = \emptyset$  then
5:   return  $\perp$ 
6: end if
7: Parse  $I$  as a multiset  $\{i_1, i_2, \dots, i_N\}$  such that  $i_1 \leq i_2 \leq \dots \leq i_N$ 
8:  $\mathcal{O} \leftarrow \{(h_{i_m}, \text{out}_{i_m})\}_{m=1}^N$ ;  $\mathcal{O}' \leftarrow \emptyset$ ;  $L \leftarrow \lceil \frac{8Nq}{\varepsilon} \ln \left( \frac{8Nn}{\varepsilon} \right) \rceil$ ;  $T_{\max} \leftarrow nL$ 
9: for  $m = 1$  to  $N$  do
10:    $\mathcal{O}'_m \leftarrow \emptyset$ ;  $c_m \leftarrow 0$ 
11:   while  $|\mathcal{O}'_m| < n$  and  $c_m < T_{\max}$  do
12:      $c_m \leftarrow c_m + 1$ ;  $h_{i_m}^{(m, c_m)}, \dots, h_q^{(m, c_m)} \xleftarrow{\$} \mathbb{H}^{q-i_m+1}$ 
13:      $Q^{(m, c_m)} \leftarrow (h_1, \dots, h_{i_m-1}, h_{i_m}^{(m, c_m)}, \dots, h_q^{(m, c_m)})$ 
14:      $(I', \{\text{out}'_i\}_{i \in I'}, \theta') \leftarrow \mathcal{A}(\text{par}, Q^{(m, c_m)}; \rho)$ 
15:     if  $i_m \in I'$  then
16:       if  $h_{i_m}^{(m, c_m)} \notin \{h_{i_m}\} \cup \{h' \mid (h', \cdot) \in \mathcal{O}'_m\}$  then
17:          $\mathcal{O}'_m \leftarrow \mathcal{O}'_m \cup \{(h_{i_m}^{(m, c_m)}, \text{out}'_{i_m})\}$ 
18:       end if
19:     end if
20:   end while
21:   if  $|\mathcal{O}'_m| < n$  then
22:     return  $\perp$ 
23:   end if
24:    $\mathcal{O}' \leftarrow \mathcal{O}' \cup \mathcal{O}'_m$ 
25: end for
26: return  $(\mathcal{O}, \mathcal{O}')$ 

```

Lemma 2 (Generalized Multi-Instance Multi-Forking Lemma). *Let $\mathbb{H}$ be a finite set and let $q, N, n \in \mathbb{N}$ with $N, n \geq 1$. Let $\mathcal{A}$ be a randomized algorithm that, on input par and a vector $h \in \mathbb{H}^q$, returns an index set I and side outputs $(\{\text{out}_i\}_{i \in I}, \theta)$. Define $\hat{\varepsilon}$ as above. Assume $|\mathbb{H}| > \frac{16Nnq}{\hat{\varepsilon}}$. Then the*

accepting probability of $\text{MIMFork}_{N,n}^A$ is

$$\text{acc}(\text{MIMFork}_{N,n}^A) = \Pr \left[\text{MIMFork}_{N,n}^A(\text{par}) \neq \perp \right] \geq \frac{\hat{\varepsilon}}{8},$$

where the probability is over $\text{par} \leftarrow \text{GrGen}(1^\lambda)$ and the coins of $\text{MIMFork}_{N,n}^A$. Moreover, whenever the algorithm returns $(\mathcal{O}, \mathcal{O}')$, for every occurrence i_m the collection $\mathcal{O}'_m$ contains n pairs whose first components are pairwise distinct and distinct from h_{i_m} .

Proof. For a fixed par , let $\varepsilon = \varepsilon(\text{par})$ and $S = S(\text{par})$ where $\Pr[S] = \varepsilon$ by definition. For every $f \in S$, let $I(f) = \{i_1(f), \dots, i_N(f)\}$, where $i_1(f) \leq \dots \leq i_N(f)$. For $j \in \{1, \dots, q\}$, let $f_j = (\rho, h_1, \dots, h_{j-1})$ denote the prefix before the j -th random-oracle response, and let $s_j = (h_j, \dots, h_q)$ denote the corresponding suffix. For every $m \in \{1, \dots, N\}$ and $j \in \{1, \dots, q\}$, define $A_m^j = \{f \in S \mid i_m(f) = j\}$ and $B_m^j = \left\{ f \in \Omega \mid \Pr_{s_j}[(f_j \| s_j) \in A_m^j] \geq \frac{\varepsilon}{2Nq} \right\}$. Let $E_m = \bigcup_{j=1}^q (A_m^j \cap B_m^j)$. By part (3) of the Splitting Lemma [5, Lemma 2], for every m ,

$$\Pr[E_m \mid S] = \sum_{j=1}^q \Pr[A_m^j \cap B_m^j \mid S] \geq \sum_{j=1}^q \left(\Pr[A_m^j \mid S] - \frac{\varepsilon/(2Nq)}{\Pr[S]} \right) = 1 - \frac{1}{2N}. \quad (1)$$

Hence, by the union bound, it gives

$$\Pr \left[S \cap \bigcap_{m=1}^N E_m \right] \geq \varepsilon \left(1 - \sum_{m=1}^N \Pr[\overline{E_m} \mid S] \right) \geq \frac{\varepsilon}{2}. \quad (2)$$

Condition on $S \cap \bigcap_{m=1}^N E_m$. Fix an occurrence m and suppose that fewer than n distinct forks have been collected. Since the original tape belongs to E_m , we have $f \in B_m^{i_m}$, and hence a fresh suffix yields an execution in $A_m^{i_m}$ with probability at least $\varepsilon/(2Nq)$. Let $H_{\text{forbid}} = \{h_{i_m}\} \cup \{h' \mid (h', \cdot) \in \mathcal{O}'_m\}$. Since $|\mathcal{O}'_m| < n$, we have $|H_{\text{forbid}}| \leq n$, and conditioned on the current state of the local rewinding process, the probability that a fresh trial produces an execution in $A_m^{i_m}$ and yields a new response is at least $p \geq \frac{\varepsilon}{2Nq} - \frac{n}{|\mathbb{H}|} > \frac{\varepsilon}{2Nq} - \frac{\hat{\varepsilon}}{16Nq} > \frac{\varepsilon}{4Nq}$, where the second inequality uses $|\mathbb{H}| > 16Nnq/\hat{\varepsilon}$, and the last uses $\varepsilon > \hat{\varepsilon}/2$ for the good parameters.

Thus, as long as fewer than n forks have been collected, every rewinding trial has conditional probability at least $\varepsilon/(4Nq)$ of producing a new valid fork. Recall that $T_{\max} = nL$, where now $L = \left\lceil \frac{8Nq}{\hat{\varepsilon}} \ln \left(\frac{8Nn}{\hat{\varepsilon}} \right) \right\rceil$. Partition the nL trials into n consecutive blocks of L trials. Since each trial has conditional success probability at least $p > \varepsilon/(4Nq) > \hat{\varepsilon}/(8Nq)$, the probability that a fixed block produces no new fork is at most $(1-p)^L \leq \exp(-pL) \leq \frac{\hat{\varepsilon}}{8Nn}$.

Therefore, let Fail_m be the event that the m -th occurrence fails to collect n distinct forks. By the union bound over the n blocks and then over the N

occurrences,

$$\Pr \left[\text{Fail}_{\text{any}} \mid S \cap \bigcap_{m=1}^N E_m \right] \leq N \cdot n \cdot \frac{\hat{\varepsilon}}{8Nn} = \frac{\hat{\varepsilon}}{8}, \quad (3)$$

where $\text{Fail}_{\text{any}} = \bigcup_{m=1}^N \text{Fail}_m$. Since we are conditioning on a good parameter with $\varepsilon > \hat{\varepsilon}/2$, we have $\hat{\varepsilon}/8 < \varepsilon/4$. Combining (2) and (3), for every par with $\varepsilon(\text{par}) > \hat{\varepsilon}/2$, we have $\text{frk}(\text{par}) \geq \frac{\varepsilon}{2} - \frac{\varepsilon}{4} = \frac{\varepsilon}{4}$.

Finally, average over $\text{par} \leftarrow \text{GrGen}(1^\lambda)$. Let $P' = \{\text{par} \mid \varepsilon(\text{par}) > \hat{\varepsilon}/2\}$. For $\text{par} \in P'$, we have $\text{frk}(\text{par}) \geq \frac{\varepsilon(\text{par})}{4}$ while for $\text{par} \notin P'$, we use the trivial bound $\text{frk}(\text{par}) \geq 0$. Therefore,

$$\begin{aligned} \text{acc} \left(\text{MIMFork}_{N,n}^A \right) &\geq \frac{1}{|P|} \sum_{\text{par} \in P'} \frac{\varepsilon(\text{par})}{4} \\ &= \frac{1}{4} \left(\hat{\varepsilon} - \frac{1}{|P|} \sum_{\text{par} \notin P'} \varepsilon(\text{par}) \right) \geq \frac{1}{4} \left(\hat{\varepsilon} - \frac{\hat{\varepsilon}}{2} \right) = \frac{\hat{\varepsilon}}{8}. \quad \square \end{aligned}$$

3 Our Constructions

In this section we present FROST2+ (shown in Figure 2) as a TS-SUF-4 variant of FROST2 from Bellare et al. [6]. Different from Bellare et al.'s ATS[TS, DS] transformation [6], which requires each signer to manage an extra digital signature key-pair to authenticate the pre-processing token ($R = g^r, S = g^s$), ours reuses only the existing signing key share and the secret exponents already presented in the token. That said, FROST2+ has the same construction as FROST2 except in the SPP and PS algorithms. The SPP now broadcast not only the pre-processing token (R, S) but also a short authentication transcript $(u, V = g^v)$ with:

$$u = v + \text{st}_i.\text{sk} \cdot e + re^2 + se^3$$

which is to be verified during PS. A signer aborts PS whenever a peer's (u, V) verification fails. The correctness follows directly from the equations such that if all parties are honest, then for each authentication transcript (u, V) , we have $g^u = VX_i^e R^{e^2} S^{e^3}$. In the signing phase, the partial signatures satisfy $z_i = r_i + ds_i + c\Delta_{\text{id}_i, S}(0)\text{st}_i.\text{sk}$. Aggregating $z = \sum_{\text{id}_i \in \mathcal{S}} z_i$ gives $g^z = \tilde{R}\tilde{X}^c$ and Vf returns 1. Moreover, as discussed in the subsequent sections, when considering a batch verification of k tokens $(R_1, S_1, \dots, R_k, S_k)$, the transcript size remains constant.

3.1 Pre-Computations

Although the TS-SUF- i notions of Bellare et al. [6] assume a designated leader for collecting and distributing pre-processing tokens and signing requests to simplify security analysis, they explicitly cover scenarios in which corrupted signers take turns acting as the leader. Consequently, a leaderless design introduces no

Setup (1^λ) $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$ $H_{\text{pp}}, H_{\text{non}}, H_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ $\text{par} \leftarrow ((\mathbb{G}, p, g), H_{\text{pp}}, H_{\text{non}}, H_{\text{sig}})$ return par	CompPar ($\tilde{X}, lr$) $M \leftarrow lr.\text{msg}$ $d \leftarrow H_{\text{non}}(\tilde{X}, lr)$ $(R_i, S_i) \leftarrow lr.\text{PP}(\text{id}_i)$ $\tilde{R} \leftarrow \prod_{\text{id}_i \in lr.S} R_i (\prod_{\text{id}_i \in lr.S} S_i)^d$ $c \leftarrow H_{\text{sig}}(\tilde{X}, M, \tilde{R})$ return $(\tilde{R}, c, d)$
Kg (par, t, ns) : For $i = 0$ to $t - 1$, do $a_i \in \mathbb{Z}_p^*$ For $i = 1$ to ns , do $\text{sk}_i \leftarrow \sum_{j=0}^{t-1} a_j \times \text{id}_i^j; X_i \leftarrow g^{\text{sk}_i}$ $\tilde{X} = g^{a_0}; \text{aux} \leftarrow (\{X_i\}_{1 \leq i \leq \text{ns}})$ For $i = 1$ to ns , do $\text{st}_i \leftarrow (\tilde{X}, \text{aux}, \text{sk}_i)$ return $\{\text{st}_i\}_{1 \leq i \leq \text{ns}}$	PS (lr, st_i) $pp_i \leftarrow lr.\text{PP}(\text{id}_i)$ If $\text{st}_i.\text{mapPP}(pp_i) = \perp$, return $(\perp, \text{st}_i)$ $\tilde{X}, \text{aux} \leftarrow \text{st}_i$ For $\text{id}_i \in lr.S$, do $(R_i, S_i, u_i, V_i) \leftarrow lr.\text{PP}(\text{id}_i)$ $e' = H_{\text{pp}}(\tilde{X}, X_i, R_i, S_i, V_i)$ If $g^{u_i} \neq V_i X_i^{e'} R_i^{e'^2} S_i^{e'^3}$, return $\perp$ Else $lr.\text{PP}(\text{id}_i) \leftarrow (R_i, S_i)$ $(r_i, s_i) \leftarrow \text{st}_i.\text{mapPP}(pp_i)$ $\text{st}_i.\text{mapPP}(pp_i) \leftarrow \perp$ $(\tilde{R}, c, d) \leftarrow \text{CompPar}(\text{st}_i.\tilde{X}, lr)$ $z_i \leftarrow r_i + d \cdot s_i + c \cdot \Delta_{\text{id}_i, S}(0) \cdot \text{st}_i.\text{sk}$ $\text{psig}_i \leftarrow (\tilde{R}, z_i)$ return $(\text{psig}_i, \text{st}_i)$
SPP (st_i) $r, s, v \xleftarrow{\$} \mathbb{Z}_p^*$ $e = H_{\text{pp}}(\tilde{X}, X_i, R = g^r, S = g^s, V = g^v)$ $u = v + \text{st}_i.\text{sk} \cdot e + r e^2 + s e^3$ $pp \leftarrow (R, S, u, V)$ $\text{st}_i.\text{mapPP}(pp) \leftarrow (r, s, v)$ return (pp, st_i)	Agg ($\{\text{psig}_i\}_{\text{id}_i \in lr.S}, \text{st}_0$) $\tilde{R} \leftarrow \perp; z \leftarrow 0$ For $\text{id}_i \in S$, do If $\tilde{R} = \perp$, $\tilde{R} \leftarrow \text{psig}_i.\tilde{R}$ If $\tilde{R} \neq \text{psig}_i.\tilde{R}$, return $(\perp, \text{st}_0)$ $z \leftarrow z + \text{psig}_i.z$ $\sigma \leftarrow (\tilde{R}, z)$ return (σ, st_0)
LPP ($\text{id}_i, pp, \text{st}_0$): $\text{st}_0.\text{currPP}_i \leftarrow \text{st}_0.\text{currPP}_i \cup \{pp\}$ return st_0	SVf ($\tilde{X}, lr, \sigma$) $(\tilde{R}, z) \leftarrow \sigma$ $(\tilde{R}', c', d) \leftarrow \text{CompPar}(\tilde{X}, lr)$ return $(\tilde{R} = \tilde{R}') \wedge (g^z = \tilde{R} \tilde{X}^{c'})$
LR (M, S, st_0) If $\exists \text{id}_i \in S : \text{st}_0.\text{curPP}(\text{id}_i) = \emptyset$, return $(\perp, \text{st}_0)$ $lr.\text{msg} \leftarrow M; lr.S \leftarrow S$ For $\text{id}_i \in S$, do Pick pp_i from $\text{st}_0.\text{curPP}(\text{id}_i)$ $lr.\text{PP}(\text{id}_i) \leftarrow pp_i$ $\text{st}_0.\text{curPP}(\text{id}_i) \leftarrow \text{st}_0.\text{curPP}(\text{id}_i) \setminus \{pp_i\}$ return (lr, st_0)	
Vf ($\tilde{X}, M, \sigma$) $(\tilde{R}, z) \leftarrow \sigma; c \leftarrow H_{\text{sig}}(\tilde{X}, M, \tilde{R})$ return $(g^z = \tilde{R} \tilde{X}^c)$	

Fig. 2. The FROST2 and FROST2+ signature schemes where FROST2 contains all but the solid box. GrGen generates cyclic group parameter in which the discrete logarithm assumption holds, st is state, $\text{ns} < p - 1$ is the number of signers, and t is the signing threshold. The function mapPP records dlogs of tokens pp , and currPP is the current pp collection.

unmodeled attack vectors. In particular, attacks captured by TS-UF-3 and TS-UF-4, where a corrupted leader forging tokens or requests, are already included in the TS-SUF- i adversary model. When an honest signer initiates the protocol, the same corruption bounds apply, preserving the original security guarantees.

If our proposed FROST2+ adopts this leaderless approach, signers can collect and verify pre-processing tokens as in the original FROST [22] prior to the partial signing (PS) phase. This pre-verification step improves runtime performance, as shown in Table 2, without compromising security.

3.2 Batch Verification

Similar to ATS[FROST,DS] [8,6] which can generate an offline signature on multiple tokens for batch verification, our FROST2+ generates an offline authentication transcript on multiple tokens for the same purpose. Specifically, when a signer id_i has $n(\geq 2)$ tokens to be verified, during SPP, the signer computes:

$$\begin{aligned} \{r_{i,j}, s_{i,j}\}_{j=1}^n, v &\stackrel{\$}{\leftarrow} \mathbb{Z}_p^*, e = H_{\text{pp}}(\tilde{X}, X_i, \{R_{i,j}, S_{i,j}\}_{j=1}^n, V = g^v) \\ u &= v + \text{st}_i \cdot \text{sk} \cdot e + \sum_{j=1}^n r_{i,j} e^{j+1} + \sum_{j=1}^n s_{i,j} e^{j+1+n}, pp \leftarrow (\{R_{i,j}, S_{i,j}\}_{j=1}^n, u, V) \end{aligned}$$

where the transcript size (u, V) remains constant. Subsequently, the verification is done by checking whether:

$$g^u = V X_i^{e'} \prod_{j=1}^n R_{i,j}^{e'^{j+1}} \prod_{j=1}^n S_{i,j}^{e'^{j+1+n}}$$

holds where $e' = H_{\text{pp}}(\tilde{X}, X_i, \{R_{i,j}, S_{i,j}\}_{j=1}^n, V)$.

Enabling this batch verification feature does not affect the TS-SUF-4 security level, but the reduction loss because the run time of reduction is linear to n as shown in Section 4. If a tighter reduction is desirable, then one may need to explore the Multi-Instance Reset Lemma [21].

4 Security Analysis

We prove the TS-SUF-4 security of FROST2+ under the OMDL assumption in the random oracle model. Our proving strategy is to separate the security analysis into three parts which handle the events

1. $E_1 : \mathbf{Vf}(\tilde{X}, M^*, \sigma^*) \wedge (lr^* = \perp \vee |S_2(lr^*)| < t - |CS|),$
2. $E_2 : \mathbf{Vf}(\tilde{X}, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge S_2(lr^*) \neq S_3(lr^*),$
3. $E_3 : \mathbf{Vf}(\tilde{X}, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge \neg(S_2(lr^*) = S_3(lr^*) = S_4(lr^*)),$

respectively. We first present the security proofs in the centralized key generation setting to ease exposition, and show how to extend them to the decentralized key generation setting in Section 4.1.

Theorem 1. *Let $\mathcal{F}$ be a TS-SUF-4 forger against FROST2+ making at most q queries to the random oracle. Suppose the success probability of $\mathcal{F}$ is bounded by $\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \Pr[E_1] + \Pr[E_2] + \Pr[E_3]$. Then, there exist three OMDL adversaries $\mathcal{B}_1, \mathcal{B}_2$ and $\mathcal{B}_3$ such that:*

$$\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \sqrt{q \cdot \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_1)} + 8\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_2) + 8\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_3) + \text{negl}$$

where $\mathcal{B}_1$ and $\mathcal{B}_3$ issue at most $2q_s + t$ queries to the CHALL oracle, while $\mathcal{B}_2$ issues at most $2q_s$ queries. Moreover, their running times are bounded above by

$$\begin{aligned} T(\mathcal{B}_1) &\leq 2T(\mathcal{F}) + T(q), \\ T(\mathcal{B}_2) &\leq \frac{16\text{ns}^2 q \ln(16\text{ns} / \Pr[E_2])}{\Pr[E_2]} T(\mathcal{F}) + T(q), \text{ and} \\ T(\mathcal{B}_3) &\leq \frac{24q \ln(24 / \Pr[E_3])}{\Pr[E_3]} \cdot T(\mathcal{F}) + T(q), \end{aligned}$$

respectively, where $t \leq \text{ns}$, and $T(q)$ denotes the time required for the relevant group operations and exponentiations.

Proof. The proof follows from Lemmas 3, 4 and 7. $\square$

Lemma 3 is intended for the event E_1 , where the forgery must contain at least one forged psig_i . The proof is similar to the TS-SUF-2 proof of FROST2 [8, Lemma 5.2], except that we add an additional $\text{RO}[\text{H}_{\text{pp}}]$ to handle the authentication transcript (u, V) and adjust the sequence of hash queries accordingly.

Lemma 3. *There exists an OMDL adversary $\mathcal{B}_1$ making at most $2q_s + t$ queries to the CHALL oracle and q queries to the random oracle such that*

$$\Pr[E_1] \leq \sqrt{q \cdot \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_1)} + \text{negl}.$$

Moreover, $\mathcal{B}_1$ runs in time roughly twice that of $\mathcal{F}$, plus the time $T(q)$ required to perform exponentiations and group operations.

Proof. Let $\mathcal{F}$ be an adversary as described in the theorem. Denote the output message-signature pair of $\mathcal{F}$ as $(M^*, \text{sig}^* = (\tilde{R}^*, z^*))$. Without loss of generality, we assume $\mathcal{F}$ always queries RO on $\text{H}_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ before $\mathcal{F}$ returns and always queries RO on $\text{H}_{\text{pp}}(\tilde{X}, X_i, R, S, V)$ and $\text{H}_{\text{non}}(\tilde{X}, lr)$ prior to the query $\text{PSIGNO}(\text{id}_i, lr)$ for some id_i and lr . This adds up to $2q_s$ additional RO queries, and we let $q = q_h + 2q_s + 1$ where $q_h = q_{\text{pp}} + q_{\text{non}} + q_{\text{sig}}$.

Let lr^* be the leader query whose nonce query $\text{H}_{\text{non}}(\tilde{X}, lr^*)$ is the last such query before the signature hash query $\text{H}_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ and which satisfies $\text{SVf}(\tilde{X}, lr^*, \sigma^*) = \text{true}$ (if none exist, $lr^* = \perp$). Denote the event E_1 as

$$\text{Vf}(\tilde{X}, M^*, \sigma^*) \wedge (lr^* = \perp \vee |S_2(lr^*)| < t - |CS|).$$

We first construct an algorithm $\mathcal{C}$ compatible with the syntax in Lemma 1. The input of $\mathcal{C}$ consists of $2q_s + t$ uniformly random group elements $\mathbf{A} =$

$\{A_0, \dots, A_{t-1}\}, \mathbf{R} = \{R_1, \dots, R_{q_s}\}, \mathbf{S} = \{S_1, \dots, S_{q_s}\} \in \mathbb{G}$ and uniformly random integers $h_1, \dots, h_{2(q-q_{pp})}, h_{2(q-q_{pp})+1}, \dots, h_{2q} \in \mathbb{Z}_p$. Also, $\mathcal{C}$ can access an oracle DLOG, which on input $X \in \mathbb{G}$ outputs $\text{DL}_{\mathbb{G},g}(X)$. To start with, $\mathcal{C}$ initializes all the states $\text{st}_0, \dots, \text{st}_{\text{ns}}$. In addition, it initializes counters $\text{ctr}_s, \text{ctr}_h$ to 0, ctr_{pp} to $2(q - q_{pp})$ and a function dt to an empty table, which are used to record the DLOG query related to each (R_j, S_j) . $\mathcal{C}$ also initializes $\text{curLR} \leftarrow \emptyset$ to record all leader requests that appear during the game and initializes ctrPP to an empty table, which are used to record the counter corresponding to each token generated by honest signers. We also use a flag BadPPO to denote whether a bad event occurs, which is initially set to false. Then, $\mathcal{C}$ runs $\mathcal{F}$ with access to the oracles $\widetilde{\text{INIT}}, \widetilde{\text{PPO}}, \widetilde{\text{PSIGNO}}, \widetilde{\text{RO}}$, which are simulated as follows.

$\widetilde{\text{INIT}}(CS)$: $\mathcal{C}$ initializes $\mathbf{H}$ to an empty table and sets $\tilde{X} \leftarrow A_0, X_i = \prod_{j=0}^{t-1} A_j^{\text{id}_i^j}$ for $i \in [1..\text{ns}]$, and $\text{st}_i.\text{sk} = \text{DLOG}(X_i)$ for $\text{id}_i \in CS$. Finally, $\mathcal{C}$ returns $\tilde{X}$, $\text{aux} = (X_1, \dots, X_{\text{ns}}), \{\text{st}_i.\text{sk}\}_{\text{id}_i \in CS}$.

$\widetilde{\text{RO}}[\text{H}_{\text{pp}}](x)$: If $\text{H}_{\text{pp}}(x) \neq \perp$, $\mathcal{C}$ returns $\text{H}_{\text{pp}}(x)$. Otherwise, $\mathcal{C}$ parses x as $(\tilde{X}, X_i, R, S, V)$.

If the parsing fails or $\tilde{X} \neq A_0$, $\mathcal{C}$ sets $\text{H}_{\text{pp}}(x) \xleftarrow{\$} \mathbb{Z}_p^*$ and returns $\text{H}_{\text{pp}}(x)$. Otherwise, $\mathcal{C}$ sets $\text{ctr}_{pp} \leftarrow \text{ctr}_{pp} + 1$ and $\text{H}_{\text{pp}}(x) \leftarrow h_{\text{ctr}_{pp}}$ then returns $\text{H}_{\text{pp}}(x)$.

$\widetilde{\text{RO}}[\text{H}_{\text{non}}](x)$: If $\text{H}_{\text{non}}(x) \neq \perp$, $\mathcal{C}$ returns $\text{H}_{\text{non}}(x)$. Otherwise, $\mathcal{C}$ parses x as $(\tilde{X}, lr)$. If the parsing fails or $\tilde{X} \neq A_0$, $\mathcal{C}$ sets $\text{H}_{\text{non}}(x) \leftarrow \mathbb{Z}_p^*$ and returns $\text{H}_{\text{non}}(x)$. Else, $\mathcal{C}$ sets $\text{ctr}_h \leftarrow \text{ctr}_h + 1$ and $\text{H}_{\text{non}}(x) \leftarrow h_{2\text{ctr}_h-1}$. It then computes $\tilde{R} \leftarrow \prod_{\text{id}_i \in lr.S} R_i S_i^{h_{2\text{ctr}_h-1}}$, where $(R_i, S_i, u_i, V_i) \leftarrow lr.\text{PP}(\text{id}_i)$. If $\text{H}_{\text{sig}}(\tilde{X}, lr.\text{msg}, \tilde{R}) = \perp$, $\mathcal{C}$ sets $\text{H}_{\text{sig}}(\tilde{X}, lr.\text{msg}, \tilde{R}) = h_{2\text{ctr}_h}$. In addition, $\mathcal{C}$ defines $\text{mapLR}(\text{ctr}_h) = lr$ and sets $\text{curLR} \leftarrow \text{curLR} \cup \{lr\}$. Finally, $\mathcal{C}$ returns $\text{H}_{\text{non}}(x)$.

$\widetilde{\text{RO}}[\text{H}_{\text{sig}}](x)$: If $\text{H}_{\text{sig}}(x) \neq \perp$, $\mathcal{C}$ returns $\text{H}_{\text{sig}}(x)$. Otherwise, $\mathcal{C}$ parses x as $(\tilde{X}, M, \tilde{R})$.

If the parsing fails or $\tilde{X} \neq X$, $\mathcal{C}$ sets $\text{H}_{\text{sig}}(x) \xleftarrow{\$} \mathbb{Z}_p^*$ and returns $\text{H}_{\text{sig}}(x)$. Else, $\mathcal{C}$ sets $\text{ctr}_h \leftarrow \text{ctr}_h + 1$ and $\text{H}_{\text{sig}}(x) \leftarrow h_{2\text{ctr}_h}$. Finally, $\mathcal{C}$ returns $\text{H}_{\text{sig}}(x)$.

$\widetilde{\text{PPO}}(\text{id}_i)$ query: Same as in the game $\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-3}}$, except that, in the simulation of SPP, $\mathcal{C}$ increments ctr_s and ctr_{pp} by 1, chooses $u_{\text{ctr}_{pp}} \xleftarrow{\$} \mathbb{Z}_p^*$ and computes

$V_{\text{ctr}_{pp}} = g^{u_{\text{ctr}_{pp}}} / X_i^{h_{\text{ctr}_{pp}}} R_{\text{ctr}_s}^{h_{\text{ctr}_{pp}}} S_{\text{ctr}_s}^{h_{\text{ctr}_{pp}}}$. If $\text{H}_{\text{pp}}(\tilde{X}, X_i, R_{\text{ctr}_s}, S_{\text{ctr}_s}, V_{\text{ctr}_h}) = \perp$, $\mathcal{C}$ sets $\text{H}_{\text{pp}}(\tilde{X}, X_i, R_{\text{ctr}_s}, S_{\text{ctr}_s}, V_{\text{ctr}_h}) \leftarrow h_{\text{ctr}_{pp}}, pp \leftarrow (R_{\text{ctr}_s}, S_{\text{ctr}_s}, u_{\text{ctr}_h}, V_{\text{ctr}_h}), \text{st}_i.\text{mapPP}(pp) \leftarrow (0, 0, 0, 0)$, and $\text{ctrPP}(\text{id}_i, pp) \leftarrow \text{ctr}_s$. Else if $\text{H}_{\text{pp}}(\tilde{X}, X_i, R_{\text{ctr}_s}, S_{\text{ctr}_s}, V_{\text{ctr}_{pp}}) \neq \perp$, BadPPO is set to true. If there exists $lr \in \text{curLR}$ such that $lr.\text{PP}(\text{id}_i) = (R_{\text{ctr}_s}, S_{\text{ctr}_s}, u'_{\text{ctr}_{pp}}, V'_{\text{ctr}_{pp}})$ for some valid $(u'_{\text{ctr}_{pp}} \neq u_{\text{ctr}_{pp}}, V'_{\text{ctr}_{pp}} \neq V_{\text{ctr}_{pp}})$, then BadPPO is also set to true.

$\widetilde{\text{PSIGNO}}(\text{id}_i, lr)$ query: Same as in the game $\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-3}}$, except that, in the simulation of PS, if $\text{st}_i.\text{mapPP}(pp) \neq \perp$, $\mathcal{C}$ sets $z_i \leftarrow \text{DLOG}(R_j S_j^{d_i} X_i^{c\Delta_{\text{id}_i, lr.S}(0)})$, where $j \leftarrow \text{ctrPP}(\text{id}_i, lr.\text{PP}(\text{id}_i))$. In addition, $\mathcal{C}$ sets $\text{dt}(j) \leftarrow (\text{id}_i, k, d_i, c\Delta_{\text{id}_i, lr.S}(0), z_i)$, where k denotes the index such that $\text{H}_{\text{non}}(\tilde{X}, lr)$ is set to h_{2k-1} during the simulation.

After receiving the output $(lr^* = \{M^*, \mathcal{S}, \text{PP}\}, \sigma^* = (\tilde{R}^*, z^*))$ from $\mathcal{F}$, $\mathcal{C}$ returns $\perp$ if $\text{BadPPO} = \text{true}$, or E_1 does not occur. Else, $\mathcal{C}$ finds the index i_{sig} such that $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ is set to $h_{i_{\text{sig}}}$ during the simulation. By our assumption on $\mathcal{F}$, we know such i_{sig} must exist. Then, $\mathcal{C}$ returns $(i_{\text{sig}}, \text{Out})$, where Out consists of all variables received or generated by $\mathcal{C}$.

Analysis of $\mathcal{C}$. To apply Lemma 1, we define $SS := \{2k\}_{k \in [1..q-q_{pp}]}$ and let GrGen be the algorithm that samples $2q_s + t$ group elements uniformly from $\mathbb{G}$ and outputs them. From the simulation, the output index i_{sig} of $\mathcal{C}$ is always in SS . Moreover, it is clear that $\mathcal{C}$ simulates the game $\mathbf{G}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}$ perfectly when all the inputs to $\mathcal{C}$ are uniformly sampled from their domain, which implies $\text{acc}(\mathcal{C}) \geq \Pr[E_1] - \Pr[\text{BadPPO}]$, where $\Pr[E_1]$ denotes the probability in the original $\mathbf{G}_{\text{FROST}_{2+}}^{\text{ts-suf-3}}$ game with $\mathcal{F}$ as in Lemma 1, and $\Pr[\text{BadPPO}]$ denotes the probability that $\text{BadPPO} = \text{true}$ at the end of $\mathcal{C}$'s execution. Since each pair (R_j, S_j) is sampled uniformly from $\mathbb{G}$ and the corresponding authentication transcript (u, V) is generated by programming $\text{RO}[H_{\text{pp}}]$, for each $\text{PPO}(\text{id}_i)$ query, the probability BadPPO is set to true is negligible and at most $q_h/p + |\text{curLR}|/p = 2q_h/p$. Therefore, we have $\Pr[\text{BadPPO}] \leq 2q_s q_h/p$. By Lemma 1,

$$\text{acc}(\text{Fork}^{\mathcal{C}}) \geq \frac{(\Pr[E_1] - 2q_s q_h/p)^2}{q - q_{pp}} \geq \frac{\Pr[E_1]^2}{q} - \frac{4\Pr[E_1]q_s q_h}{p \cdot q} \geq \frac{\Pr[E_1]^2}{q} - \frac{2q_h}{p}.$$

Construct $\mathcal{B}_1$ from $\text{Fork}^{\mathcal{C}}$. We now construct the OMDL adversary $\mathcal{B}_1$ using $\text{Fork}^{\mathcal{C}}$ and the available DLOG oracle. To begin, $\mathcal{B}_1$ queries INIT and then queries CHALL oracle $2q_s + t$ times to generate the three lists of group elements $(\mathbf{A}, \mathbf{R}, \mathbf{S})$ as the input to run $\text{Fork}^{\mathcal{C}}$. Without loss of generality, we can assume all the OMDL challenges are distinct, since otherwise $\mathcal{B}_1$ can solve them trivially. All DLOG queries from $\text{Fork}^{\mathcal{C}}$ are forwarded by $\mathcal{B}_1$ to the DLOG oracle in the game $\mathbf{G}_{\mathbb{G}}^{\text{omdl}}$. Denote by BadHash the event that any two of the scalars $h_1, h'_1, \dots, h_q, h'_q$ generated in the execution of $\text{Fork}^{\mathcal{C}}$ are equal. Since these scalars are sampled uniformly from $\mathbb{Z}_p$, we have $\Pr[\text{BadHash}] \leq 2q^2/p$.

It remains to show that if $\text{Fork}^{\mathcal{C}}$ returns $(i_{\text{sig}}, \text{Out}, \text{Out}')$ and BadHash does not occur, $\mathcal{B}_1$ wins the game $\mathbf{G}_{\mathbb{G}}^{\text{omdl}}$. Absorbing all strictly negligible terms into a function negl , we have

$$\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_1) \geq \text{acc}(\text{Fork}^{\mathcal{C}}) - \Pr[\text{BadHash}] \geq \frac{\Pr[E_1]^2}{q} - \frac{3q^2}{p} = \frac{\Pr[E_1]^2}{q} - \text{negl}.$$

We use the notations introduced in the description of $\mathcal{C}$ to denote the variables in Out and append a prime $(\cdot)'$ to denote the variables in Out' . By the execution of $\text{Fork}^{\mathcal{C}}$, we have $(\tilde{X}, M^*, \tilde{R}^*) = (\tilde{X}', M^{*'}, \tilde{R}^{*'})$ and $\tilde{X} = A_0$. Since $i_{\text{sig}} \in SS$, let $k^* = i_{\text{sig}}/2$ where $\text{mapLR}(k^*) = lr^*$. Furthermore, if $\text{mapLR}(k^*) = \perp$, then $lr^* = \perp$ as well.

We first show how to compute the discrete logarithm of $A_0, \dots, A_{t-1}$. Denote these discrete logarithms by $a_0, \dots, a_{t-1}$ and define $f(x) := \sum_{i=0}^{t-1} a_i x^i$. Since BadHash does not occur, we have $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*) = h_{i_{\text{sig}}} \neq h'_{i_{\text{sig}}} =$

$H_{\text{sig}}'(\tilde{X}, M^*, \tilde{R}^*)$. As $g^{z^*} = \tilde{R}^* g^{a_0 h_{i_{\text{sig}}}}$ and $g^{z^{*'}} = \tilde{R}^* g^{a_0 h'_{i_{\text{sig}}}}$, $\mathcal{B}_1$ computes $f(0) = a_0 = \frac{z^* - z^{*'}}{h_{i_{\text{sig}}} - h'_{i_{\text{sig}}}}$. Define $T_{\text{dt}} := \{j : (i, k, d, c, z) \leftarrow \text{dt}(j), k = k^*\}$.

For each $j \in T_{\text{dt}} \cap T_{\text{dt}'}$, let $(i, k, d, c, z) \leftarrow \text{dt}(j)$ and $(i', k', d', c', z') \leftarrow \text{dt}'(j)$, then $g^z = R_j S_j^d X_i^c$ and $g^{z'} = R_j S_j^{d'} X_i^{c'}$.

Since $\text{BadPPO} = \text{false}$ during both executions of $\mathcal{C}$, we know that (R_j, S_j, u_j, V_j) was returned by a query $\text{PPO}(\text{id}_i)$ prior to the query $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ in the first execution of $\mathcal{C}$. Because the two executions of $\mathcal{C}$ are identical up to the query $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$, we have $\text{id}'_i = \text{id}_i$. Moreover, we know $d = h_k = h_{k^*} = h_{k'} = d'$. Therefore, $\mathcal{B}_1$ can compute $f(\text{id}_i) = \text{DL}_{\mathbb{G}, g}(X_i) = \frac{z - z'}{c - c'}$.

Let $D := \{\text{id}_i\}_{j \in T_{\text{dt}} \cap T_{\text{dt}'}, (i, k, d, c, z) \leftarrow \text{dt}(j)}$. If $S_2(lr^*) < t - |CS|$ occurs in the first execution of $\mathcal{C}$, then $|T_{\text{dt}}| = |S_2(lr^*)| < t - |CS|$. Therefore, we have $|D| = |T_{\text{dt}} \cap T_{\text{dt}'}| < t - |CS|$. Consequently, $\mathcal{B}_1$ can pick an arbitrary set $D' \subseteq HS \setminus D$ of size $(t - |CS| - |T_{\text{dt}} \cap T_{\text{dt}'}| - 1)$ and for each $\text{id}_i \in D'$, $\mathcal{B}_1$ queries the DLOG oracle on X_i . Thus $\mathcal{B}_1$ knows the value of $f(\text{id}_i)$ for every $\text{id}_i \in CS \cup D \cup D' \cup \{0\}$. Since $|CS \cup D \cup D' \cup \{0\}| = t$, $\mathcal{B}_1$ can recover $a_0, \dots, a_{t-1}$ by Lagrange interpolation.

We now show how to compute the discrete log of $R_1, S_1, \dots, R_{q_s}, S_{q_s}$. Denote these discrete logarithms by $r_1, s_1, \dots, r_{q_s}, s_{q_s}$. From the execution of $\mathcal{C}$, we know $\text{dt}(j) = (i, k, d, c, z) \neq \perp$ if and only if $\mathcal{C}$ queries DLOG on $R_j S_j^d X_i^c$. Therefore, we denote the DLOG query on $R_j S_j^d X_i^c$ as the DLOG query associated with $\text{dt}(j)$. For each $j \in [1..q_s]$, there are the following cases.

Case 0: Both $\text{dt}(j)$ and $\text{dt}'(j)$ are $\perp$. In this case, $\mathcal{B}_1$ queries $r_j \leftarrow \text{DLOG}(R_j)$ and $s_j \leftarrow \text{DLOG}(S_j)$.

Case 1: Exactly one of $\text{dt}(j)$ and $\text{dt}'(j)$ is not $\perp$. Without loss of generality, assume $\text{dt}(j) = (i, k, d, c, z)$, which implies $g^z = R_j S_j^d X_i^c$. $\mathcal{B}_1$ queries $s_j \leftarrow \text{DLOG}(S_j)$ and computes $r_j = z - d \cdot s_j - c \cdot f(\text{id}_i)$.

For all the following cases, both $\text{dt}(j)$ and $\text{dt}'(j)$ are not $\perp$ and we denote $(i, k, d, c, z) \leftarrow \text{dt}(j)$ and $(i', k', d', c', z') \leftarrow \text{dt}'(j)$.

Case 2: $k \neq k'$ or $k = k' > k^*$. In this case, we have $d = h_k \neq h_{k'} = d'$ and $g^z = R_j S_j^d X_i^c$, $g^{z'} = R_j S_j^{d'} X_i^{c'}$. Therefore, $\mathcal{B}_1$ computes $s_j = \frac{z - c f(\text{id}_i) - z' + c' f(\text{id}_i)}{d - d'}$ and $r_j = z - d \cdot s_j - c \cdot f(\text{id}_i)$.

Case 3: $k = k' = k^*$. In this case, $\mathcal{B}_1$ computes r_j, s_j the same as Case 1.

Case 4: $k = k' < k^*$. $\mathcal{B}_1$ computes r_j, s_j the same as Case 1. Also, in this case we have $d = d'$ and $c = c'$. Therefore, $\mathcal{B}_1$ queries the DLOG oracle once in order to simulate the DLOG queries associated with $\text{dt}(j)$ and $\text{dt}'(j)$.

We now count the number of DLOG queries made by $\mathcal{B}_1$. $\mathcal{B}_1$ queries the DLOG oracle $|CS|$ times to simulate the queries $\text{DLOG}(X_i)$ made by $\mathcal{C}$ for each $i \in CS$. If $S_2(lr^*) < t - |CS|$ occurs, $\mathcal{B}_1$ queries the DLOG oracle $|D'|$ times to compute $a_0, \dots, a_{t-1}$. For each $j \in [1..q_s]$, $\mathcal{B}_1$ queries DLOG twice to simulate the queries associated with $\text{dt}(j)$ and $\text{dt}'(j)$ and to compute r_j, s_j in cases 0, 1, 2, 4, and three times in **Case 3**. Since the condition of **Case 3** is equivalent to $j \in T_{\text{dt}} \cap T_{\text{dt}'}$, the total DLOG queries made by $\mathcal{B}_1$ is $2q_s + |T_{\text{dt}} \cap T_{\text{dt}'}| + |CS| + |D'| = 2q_s + t - 1$. Therefore, $\mathcal{B}_1$ breaks the OMDL assumption. $\square$

Lemma 4 below is devoted to the event E_2 , which captures forgeries under the condition $S_2(lr) \neq S_3(lr)$. Since $\text{st.mapPP}(pp_i) \neq \perp$ is enforced in the partial-signing algorithm PS, an honest signer generates a pp_i (and $S_3(lr)$ grows) whenever a $psig_i$ is requested (and $S_2(lr)$ grows), so forgeries with $S_3(lr) \subset S_2(lr)$ will not occur. Therefore, we only consider the case $S_2(lr) \subset S_3(lr)$. In other words, E_2 captures the attack in which at least one pp_i is honestly generated but the corresponding $psig_i$ is forged. Our proof idea is to embed OMDL challenges into the honest signer's pp_i , which would fall in $S_3(lr) \setminus S_2(lr)$ when E_2 occurs. We show that by using the Lemma 2 with an approach inspired by the mix forking technique [11], all secret exponents in the forger's token authentication transcripts (u, V) (i.e., those belonging to $lr^*.S \setminus S_3(lr^*)$) can be recovered, and an OMDL challenge in $S_3(lr^*) \setminus S_2(lr^*)$ can be solved without querying the DLOG oracle.

Lemma 4. *There exists an OMDL adversary $\mathcal{B}_2$ issuing at most $2q_s$ queries to the CHALL oracle and q queries to random oracle such that*

$$\Pr[E_2] \leq 8\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_2) + \text{negl}.$$

Moreover, $\mathcal{B}_2$ runs in time $T(\mathcal{B}_2) \leq \frac{16\text{ns}^2 q \ln(16\text{ns}/\Pr[E_2])}{\Pr[E_2]} \cdot T(\mathcal{F}) + T(q)$ where ns is the number of signers, and $T(q)$ is the time required to perform exponentiations and group operations.

Proof. Let $\mathcal{F}$ be an adversary as described in the theorem. Denote the output message-signature pair of $\mathcal{F}$ as $(M^*, \text{sig}^* = (\tilde{R}^*, z^*))$. Without loss of generality, we assume $\mathcal{F}$ always queries RO on $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ before $\mathcal{F}$ returns and always queries RO on $H_{\text{pp}}(\tilde{X}, X_i, R, S, V)$ and $H_{\text{non}}(\tilde{X}, lr)$ prior to the query $\text{PSIGNO}(\text{id}_i, lr)$ for some id_i and lr . This adds up to $2q_s$ additional RO queries, and we let $q = q_h + 2q_s + 1$ where $q_h = q_{\text{pp}} + q_{\text{non}} + q_{\text{sig}}$.

Let lr^* be the leader query whose nonce query $H_{\text{non}}(\tilde{X}, lr^*)$ is the last such query before the signature hash query $H_{\text{sig}}(\tilde{X}, M^*, \tilde{R}^*)$ and which satisfies $\text{SVf}(\tilde{X}, lr^*, \sigma^*) = \text{true}$. Denote the event E_2 as

$$E_2 : \forall (vk, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge S_2(lr^*) \subset S_3(lr^*).$$

To begin, we construct an algorithm $\mathcal{C}$ which is compatible with the syntax in Lemma 2. The input of $\mathcal{C}$ consists of $2q_s$ random elements $\mathbf{R} = \{R_1, \dots, R_{q_s}\}$, $\mathbf{S} = \{S_1, \dots, S_{q_s}\}$ from $\mathbb{G}$, and uniformly random integers $h_1, \dots, h_q \in \mathbb{Z}_p^*$. Also, $\mathcal{C}$ has access to an oracle DLOG, which on input $X \in \mathbb{G}$ outputs $\text{DL}_{\mathbb{G},g}(X)$. $\mathcal{C}$ initializes the signers' states $\text{st}_0, \dots, \text{st}_{\text{ns}}$, counters $\text{ctr}_h = 0$ and $\text{ctr}_s = 0$. It then runs $\mathcal{F}$, answering its oracle queries as follows.

- $\widetilde{\text{INIT}}(CS)$: $\mathcal{C}$ picks $a_0, \dots, a_{t-1} \xleftarrow{\$} \mathbb{Z}_p$, sets $f(x) = \sum_{i=0}^{t-1} a_i x^i$, and defines $\tilde{X} = g^{f(0)}$, $X_i = g^{f(\text{id}_i)}$ for $i \in [1..\text{ns}]$. For $\text{id}_i \in CS$ it stores $\text{st}_i.\text{sk} = f(\text{id}_i)$. It returns $(\tilde{X}, (X_1, \dots, X_{\text{ns}}), \{\text{st}_i.\text{sk}\}_{\text{id}_i \in CS})$.
- $\widetilde{\text{RO}}[H_{\text{pp}}](x)$: If $H_{\text{pp}}(x) \neq \perp$, $\mathcal{C}$ returns $H_{\text{pp}}(x)$. Otherwise, increments ctr_h to set $H_{\text{pp}}(x) \leftarrow h_{\text{ctr}_h}$ and returns $H_{\text{pp}}(x)$.

- $\widetilde{\text{RO}}[\text{H}_{\text{non}}](x)$: If $\text{H}_{\text{non}}(x) \neq \perp$, $\mathcal{C}$ returns $\text{H}_{\text{non}}(x)$. Otherwise, increments ctr_h to set $\text{H}_{\text{non}}(x) \leftarrow h_{\text{ctr}_h}$ and returns $\text{H}_{\text{non}}(x)$.
- $\widetilde{\text{RO}}[\text{H}_{\text{sig}}](x)$: If $\text{H}_{\text{sig}}(x) \neq \perp$, $\mathcal{C}$ returns $\text{H}_{\text{sig}}(x)$. Otherwise, increments ctr_h to set $\text{H}_{\text{non}}(x) \leftarrow h_{\text{ctr}_h}$ and returns $\text{H}_{\text{non}}(x)$.
- $\widetilde{\text{PPO}}(\text{id}_i)$: Same as in the game $\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-3}}$, except that, in the simulation of SPP, $\mathcal{C}$ selects $u \xleftarrow{\$} \mathbb{Z}_p$, increments ctr_h and ctr_s to set $e = h_{\text{ctr}_h}$, and computes $V = g^u / X_i^e R_{\text{ctr}_s}^{e^2} S_{\text{ctr}_s}^{e^3}$. $\mathcal{C}$ sets $\text{H}_{\text{pp}}(\tilde{X}, X_i, R_{\text{ctr}_s}, S_{\text{ctr}_s}, V) \leftarrow e$ and adds $pp_i \leftarrow (R_{\text{ctr}_s}, S_{\text{ctr}_s}, u, V)$ to $\text{st}_i.\text{curPP}$.
- $\widetilde{\text{PSIGNO}}(\text{id}_i, lr)$: Same as in the game $\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-3}}$, except that, in the simulation of PS, if $\text{st}_i.\text{mapPP}(pp_{\text{id}_i}) \neq \perp$ and all $pp_j \in lr.\text{PP}$ pass the verifications, $\mathcal{C}$ retrieves $(R, S) \leftarrow pp_{\text{id}_i}$ and other $(R_j, S_j) \in lr.\text{PP}$ to compute $\tilde{R} = \prod_{i \in lr.S} R_i (\prod_{i \in lr.S} S_i)^d$ where $d \leftarrow \text{H}_{\text{non}}(\tilde{X}, lr)$. $\mathcal{C}$ then removes pp_{id_i} from $\text{st}_i.\text{curPP}$ and returns $z_i \leftarrow \text{DLOG}(RS^d) + c\Delta_{\text{id}_i, lr.S}(0)f(\text{id}_i)$ where $c \leftarrow \text{H}_{\text{sig}}(\tilde{X}, lr.\text{msg}, \tilde{R})$.

Finally, $\mathcal{F}$ outputs its forgery $(lr^* = \{M^*, \mathcal{S}, \text{PP}\}, \sigma^* = (\tilde{R}^*, z^*))$ and wins the game $\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-4}}$. $\mathcal{C}$ returns $(I, \{\text{out}_i\}_{i \in I}, \theta)$ where I are the set of random oracle indices such that out_i contains $h_i \leftarrow \text{H}_{\text{pp}}(\tilde{X}, X_i, R_i, S_i, V_i)$ for $\text{id}_i \in lr^*.S \setminus S_3(lr^*)$ and $\theta = (lr^*, \sigma^*)$.

Analysis of $\mathcal{C}$. There are two possible forging cases under the condition $S_2(lr^*) \subset S_3(lr^*)$: (1) $lr^*.S \setminus S_3(lr^*) = \emptyset$; and (2) $lr^*.S \setminus S_3(lr^*) \neq \emptyset$. We first present the analysis for case (1) and subsequently explain how to extend it to handle case (2).

Case (1): When $lr^*.S \setminus S_3(lr^*) = \emptyset$, $\mathcal{F}$ forges without generating any pre-processing tokens such that $S_3(lr^*) = S_4(lr^*) = lr^*.S$. We show that there is an OMDL adversary $\mathcal{B}_2$ which uses $\mathcal{C}$ as the helper algorithm to compute the discrete logarithms of (R^*, S^*) for an $\text{id}^* \in S_3(lr^*) \setminus S_2(lr^*)$. $\mathcal{B}_2$ queries $2q_s$ challenges $\mathbf{R} = \{R_1, \dots, R_{q_s}\}$, $\mathbf{S} = \{S_1, \dots, S_{q_s}\}$ from CHALL and selects random $h_1, \dots, h_q \in \mathbb{Z}_p^*$ as the input to $\mathcal{C}$. The DLOG queries from $\mathcal{C}$ are relayed by $\mathcal{B}_2$ to its DLOG of $\mathbf{G}_{\mathbb{G}}^{\text{omdl}}$. From $(I, \{\text{out}_i\}_{i \in I}, \theta) \leftarrow \mathcal{C}$, $\mathcal{B}_2$ needs only θ to obtain $((z^*, c^*, f(0)), \tilde{R}^* = \prod_{\text{id}_j \in lr^*.S} R_j (\prod_{\text{id}_j \in lr^*.S} S_j)^{d^*})$ and compute the sum $\alpha = \sum_{\text{id}_j \in lr^*.S} r_j + d^* \cdot s_j = z^* - c^* f(0)$.

Subsequently, for all (R_j, S_j) belong to the set $D = S_3(lr^*) \setminus S_2(lr^*)$, $\mathcal{B}_2$ queries $\{r_j \leftarrow \text{DLOG}(R_j)\}_{\text{id}_j \in D}$ and $\{s_j \leftarrow \text{DLOG}(S_j)\}_{\text{id}_j \in D \setminus \{\text{id}^*\}}$ where $\text{id}^* \in D$ is the targeted identity who owns (R^*, S^*) . For those (R_j, S_j) belong to $\text{id}_j \in S_2(lr^*)$, they have been queried to PSIGNO and $\mathcal{B}_2$ knows $z_j = r_j + d^* \cdot s_j + c^* \Delta_{\text{id}_j, lr^*.S}(0)f(\text{id}_j)$ in which $\mathcal{B}_2$ can query $s_j \leftarrow \text{DLOG}(S_j)$ and computes $r_j = z_j - c^* \Delta_{\text{id}_j, lr^*.S}(0)f(\text{id}_j) - d^* \cdot s_j$. Therefore, $lr^*.S = D \cup S_2(lr^*)$ and $\mathcal{B}_2$ can extract $s^* \leftarrow \text{DL}_{\mathbb{G}, g}(S^*)$ by computing:

$$s^* = \frac{\alpha - \sum_{\text{id}_j \in lr^*.S} r_j - d^* \sum_{\text{id}_j \in lr^*.S \setminus \{\text{id}^*\}} s_j}{d^*}.$$

For the OMDL challenges not involved in the forgery $(R_j, S_j) \notin lr^*.\text{PP}(\text{id}_j)$ where $\text{id}_j \in HS$, they are either queried or have not queried to PSIGNO. If

queried, $\mathcal{B}_2$ knows the corresponding partial signature z_j in which it can query $s_j \leftarrow \text{DLOG}(S_j)$ and compute $r_j = z_j - f(\text{id}_j)c_j\Delta_{\text{id}_j,lr.S}(0) - d_js_j$. Else, $\mathcal{B}_2$ queries $r_j \leftarrow \text{DLOG}(R_j)$ and $s_j \leftarrow \text{DLOG}(S_j)$.

We now count the number of DLOG queries made by $\mathcal{B}_2$. For each challenge pair in $\{R_j = g^{r_j}, S_j = g^{s_j}\}_{j=1}^{q_s}$, $\mathcal{B}_2$ queries DLOG twice except the $s^* \leftarrow \text{DL}_{\mathbb{G},g}(S^*)$ used by the targeted honest signer $\text{id}^* \in S_3(lr^*) \setminus S_2(lr^*)$. Hence the total DLOG queries made by $\mathcal{B}_2$ equals $2q_s - 1$, breaking the OMDL assumption. Denote **BadHash** as the event of $h_i = h_j$ for some $i \neq j \leq q$. Since every hash value is selected independently, it is $\Pr[\text{BadHash}] = 1/p$. Absorbing all strictly negligible terms into a function negl , we have:

$$\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_2) \geq \Pr[\mathbf{G}_{\text{FROST}2+}^{\text{ts-suf-3}}(\mathcal{F}) = \text{true}] - \Pr[\text{BadHash}] \geq \Pr[E_2] - \text{negl},$$

and $T(\mathcal{B}_2) \approx T(\mathcal{F}) + T(q)$.

Case (2): Now, we consider the case where $\mathcal{F}$ forges with at least one pre-processing token in $lr^*.PP$ generated by itself, i.e., $lr^*.S \setminus S_3(lr^*) \neq \emptyset$. In this case, the output of $\mathcal{C}$'s single run is not sufficient for solving the OMDL problem. This is because $\mathcal{C}$ does not know the discrete logarithms of $\{R_i, S_i\}_{\text{id}_i \in lr^*.S \setminus S_3(lr^*)}$ generated by $\mathcal{F}$. To overcome this obstacle, we need a helper algorithm $\mathcal{C}'$.

Construct $\mathcal{C}'$ from $\text{MIMFork}_{N,2}^{\mathcal{C}}$. We now construct the helper algorithm $\mathcal{C}'$, which is a syntactically valid OMDL adversary, using $\text{MIMFork}_{N,2}^{\mathcal{C}}$ with $N = |I| = |lr^*.S \setminus S_3(lr^*)|$. $\mathcal{C}'$ initiates $\mathcal{C}$ the same as $\mathcal{B}_2$ did in the previous case. Similar to the proof in case (1), from $(\mathcal{O}, \mathcal{O}') \leftarrow \text{MIMFork}_{N,2}^{\mathcal{C}}$, $\mathcal{C}'$ also computes the values $(\alpha, \{r_j \leftarrow \text{DLOG}(R_j)\}_{\text{id}_j \in S_3(lr^*)}, \{s_j \leftarrow \text{DLOG}(R_j)\}_{\text{id}_j \in S_3(lr^*) \setminus \{\text{id}^*\}})$ but now $\mathcal{C}'$ needs to additionally extract $(r_i \leftarrow \text{DL}_{\mathbb{G},g}(R_i), s_i \leftarrow \text{DL}_{\mathbb{G},g}(S_i))$ from $\{(e_{i,j} = h_{i,j}, r_i e_{i,j}^2 + s_i e_{i,j}^3 = u_{i,j} - f(\text{id}_i)e_{i,j}, V_i) \leftarrow pp_{i,j}\}_{j=0}^2$ for $\text{id}_i \in D$ where $D = lr^*.S \setminus S_3(lr^*)$. As $lr^*.S = D \cup S_3(lr^*)$, $\mathcal{C}'$ can compute

$$\text{DL}_{\mathbb{G},g}(S^*) \rightarrow s^* = \frac{\alpha - \sum_{\text{id}_i \in lr^*.S} r_i - d^* \sum_{\text{id}_i \in lr^*.S \setminus \{\text{id}^*\}} s_i}{d^*}.$$

$\mathcal{C}'$ then solves those OMDL challenges not involved in the forgery by querying the DLOG oracle. Since a hash collision occurs with a negligible probability only, by Lemma 2, we have:

$$\text{acc}(\mathcal{C}') \geq \text{acc}(\text{MIMFork}_{N,2}^{\mathcal{C}}) - \text{negl} \geq \Pr[E_2]/8 - \text{negl}.$$

However, after broadcasting a pp_i in any of the $|I| \cdot T_{\max}$ rewinds in $\text{MIMFork}_{N,2}^{\mathcal{C}}$, $\mathcal{F}$ still can issue PSIGNO queries till it forges a signature. So, $\mathcal{C}'$ does not solve OMDL because the $|I| \cdot T_{\max}$ rewinds make $\mathcal{C}$ issues more than $2q_s$ DLOG queries to answer these additional PSIGNO queries.

Construct $\mathcal{B}_2$ from $\mathcal{C}'$. We now construct an OMDL adversary $\mathcal{B}_2$ which is defined the same as $\mathcal{C}'$ except that, starting from the first rewind of each instance $i \in I$, $\mathcal{B}_2$ aborts any executions of $\mathcal{C}$ after it has obtained the pp_i . This avoids $\mathcal{B}_2$

from issuing too many DLOG queries as done by $\mathcal{C}'$. Moreover, $\mathcal{B}_2$ has the same success probability as $\mathcal{C}'$ because the θ required to compute α is available after the first run ended completely, that is, before a rewind begins. Since $N = |I| = |lr^* \cdot \mathcal{S} \setminus S_3(lr^*)| \leq ns - 1$, and every $i \in I$ collected $n = 2$ forks, $\mathcal{B}_2$ uses time:

$$\begin{aligned} T(\mathcal{B}_2) &\leq T(\mathcal{C}') \leq |I| \cdot T_{max} \cdot T(\mathcal{C}) + T(q) \leq |I| \cdot T_{max} \cdot T(\mathcal{F}) + T(q) \\ &\leq ns \cdot \frac{8 \cdot ns \cdot 2q \ln(8 \cdot ns \cdot 2 / \Pr[E_2])}{\Pr[E_2]} \cdot T(\mathcal{F}) + T(q) \\ &\leq \frac{16ns^2 q \ln(16ns / \Pr[E_2])}{\Pr[E_2]} \cdot T(\mathcal{F}) + T(q). \end{aligned}$$

This completes the proof. $\square$

Up to this point, FROST2+ has achieved the TS-SUF-3 security whose forgeries are covered by the events E_1 and E_2 . If we apply the ATS[TS,DS] transformation framework as in FROST [8,6], we obtain an ATS[FROST2+, DS] scheme that achieves TS-SUF-4 security. However, the transformation comes with the cost of managing an extra DS key pair, longer pp_i and slower signing time. To avoid these, we give a direct proof instead, which is the proof for event E_3 . As $S_4(lr) = HS \cap lr^* \cdot \mathcal{S}$ and the event E_2 has covered $S_2(lr) \neq S_3(lr)$, we only need to deal with the forgeries under $S_2(lr) = S_3(lr) \subset S_4(lr)$ in E_3 .

Before presenting the direct proof, we would like to introduce a leader request forger $\mathcal{F}_{lr}$ of FROST2+ which possesses the same ability as that of a normal TS-SUF-4 forger $\mathcal{F}$, but instead of producing a valid signature (σ^*, lr^*) as the output, $\mathcal{F}_{lr}$ produces a valid leader request lr^* as the output. Denote $A \implies B$ as A implies B , that is, B is reducible to A . Below, we show that $\mathcal{F}_{lr} \iff \mathcal{F}$ when a forgery falls under $S_3(lr) \subset S_4(lr)$, that is, when at least a pre-processing token is forged.

Theorem 2 ($\mathcal{F}_{lr} \iff \mathcal{F}$). *For FROST2+, a TS-SUF-4 forger $\mathcal{F}$ which output a forgery that falls under the event*

$$E : \forall (\tilde{X}, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge S_3(lr) \subset S_4(lr).$$

is equivalent to a leader request forger $\mathcal{F}_{lr}$.

Proof. The proof follows from Lemmas 5 and 6 which show that $\mathcal{F}$ and $\mathcal{F}_{lr}$ are equivalent under E such that they are reducible to each other. $\square$

Lemma 5 ($\mathcal{F}_{lr} \implies \mathcal{F}$). *Let $\mathcal{F}$ be a TS-SUF-4 forger for FROST2+, and $\mathcal{F}_{lr}$ be a leader request forger in FROST2+. There exists an $\mathcal{F}_{lr}$ which wins $\mathbf{G}_{FROST2+}^{ts-lr}$ such that*

$$\text{Adv}_{FROST2+}^{\text{ts-suf-4}}(\mathcal{F}) = \text{Adv}_{FROST2+}^{\text{ts-lr}}(\mathcal{F}_{lr})$$

where the running time of $\mathcal{F}_{lr}$ is at most that of $\mathcal{F}$.

Proof. It is straight forward to see that $\mathcal{F}_{lr}$ can simulate the environment perfectly for $\mathcal{F}$ by acting as the middle-man in between its underlying FROST2+

oracles and $\mathcal{F}$. When $\mathcal{F}$ outputs a valid TS-SUF-4 forgery (σ^*, lr^*) , $\mathcal{F}_{lr}$ uses lr^* as its output. In fact, $\mathcal{F}_{lr}$ can return lr^* before answering the PSIGNO query on lr^* , that is, whenever it sees that a valid lr^* contains pp_i such that $S_3(lr^*) \subset S_4(lr^*)$. Therefore, $\mathcal{F}_{lr}$ runs in time at most that of $\mathcal{F}$. $\square$

Lemma 6 ($\mathcal{F} \implies \mathcal{F}_{lr}$). *Let $\mathcal{F}$ be a TS-SUF-4 forger for FROST_{2+} , and $\mathcal{F}_{lr}$ be a leader request forger in FROST_{2+} . There exists an $\mathcal{F}$ which wins $\mathbf{G}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}$ such that*

$$\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr}) \leq \Pr[E] \leq 8 \cdot \text{Adv}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}(\mathcal{F}).$$

Moreover, $\mathcal{F}$ runs in time $T(\mathcal{F}) \leq \frac{24q \ln(24/\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr}))}{\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr})} \cdot T(\mathcal{F}_{lr}) + T(q)$ where q is the number of hash queries issued and $T(q)$ is the time required to perform exponentiations and group operations. The number of $\mathcal{F}$'s queries to each oracle is at most that of $\mathcal{F}_{lr}$.

Proof. We show that if there exists a leader request forger $\mathcal{F}_{lr}$, then we can construct a TS-SUF-4 forger $\mathcal{F}$ to win in $\mathbf{G}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}$ such that the output forgery (lr^*, σ^*) falls under the event:

$$E : \forall (\tilde{X}, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge S_3(lr) \subset S_4(lr).$$

$\mathcal{F}$ constructs a wrapper $\mathcal{C}$ whose task is only to relay all oracle queries from $\mathcal{F}_{lr}$ to $\mathcal{F}$, which then replies by querying its own oracles. To begin, $\mathcal{F}$ forwards the $\text{INIT}(CS)$ query from $\mathcal{C}$ to its underlying INIT oracle and returns the required parameter $(\tilde{X}, \text{aux}, \{x_i\}_{\text{id}_i \in CS})$. Without loss of generality, we assume $|CS| = t - 1$. If $|CS| < t - 1$, $\mathcal{F}$ fills it with random $\text{id}_i \in HS$ to make sure $|CS| = t - 1$ before querying INIT but does not return the signing key share of the fillers to $\mathcal{C}$.

When $\mathcal{F}_{lr}$ returns a valid $lr^* = (M, \mathcal{S}, \{pp_i = (R_i, S_i, u_i, V_i)\}_{\text{id}_i \in \mathcal{S}})$ during the first run, $\mathcal{C}$ knows $(I, \text{out}_{i^*}, \theta = \perp)$ with $I = \{i^*\} = S_4(lr^*) \setminus S_3(lr^*)$ corresponds to a query $e_{i^*} = \text{H}_{\text{pp}}(\tilde{X}, X_{i^*}, R_{i^*}, S_{i^*}, V_{i^*})$ such that $pp_{i^*} \in lr^*$. If $1 < |I| \leq \text{ns}$, $\mathcal{C}$ selects a random index i^* from the $|I|$ indices. Then $\mathcal{C}$ returns $(e_{i^*}, \text{out}_{i^*})$.

$\mathcal{F}$ executed $(\mathcal{O}, \mathcal{O}') \leftarrow \text{MIMFork}_{1,3}^{\mathcal{C}}$ to get four transcripts

$$(\tilde{X}, X_{i^*}, R_{i^*}, S_{i^*}, V_{i^*}, u_{i^*}, e_{i^*}) \leftarrow \mathcal{O}, (\{u_{i^*,j}, e_{i^*,j}\}_{j=1}^3) \leftarrow \mathcal{O}'$$

which yields a solvable system of linear equations and allows $\mathcal{F}$ to extract $\text{st}_{i^*}.\text{sk} = x_{i^*} \leftarrow \text{DL}_{\mathbb{G},g}(X_{i^*}), r_{i^*} \leftarrow \text{DL}_{\mathbb{G},g}(R_{i^*}), s_{i^*} \leftarrow \text{DL}_{\mathbb{G},g}(S_{i^*})$. Thus, $\mathcal{F}$ knows $|CS| + 1 = t$ pairs of $(\text{id}_i, \text{st}_i.\text{sk} = x_i = f(\text{id}_i))$ and can extract the shared signing key $f(0)$ through Lagrange interpolation. $\mathcal{F}$ then utilizes $f(0)$ to generate a signature σ^* on lr^* and output (σ^*, lr^*) to win its $\mathbf{G}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}$ game.

Throughout the game, all q hash values are produced by $\mathcal{F}$'s hash oracles and we assume hash collision does not occur, or $\mathcal{F}$ can win without $\mathcal{F}_{lr}$. Since $\mathcal{F}$ simulates the environment perfectly, by Lemma 2, we have:

$$\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}(\mathcal{F}) = \text{acc}(\text{MIMFork}_{1,3}^{\mathcal{C}}) \geq \frac{\Pr[E]}{8} = \frac{\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr})}{8}$$

and $\mathcal{F}$ runs in time $T(\mathcal{F}) \leq \frac{24q \ln(24/\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr}))}{\text{Adv}_{\text{FROST}_{2+}}^{\text{ts-lr}}(\mathcal{F}_{lr})} \cdot T(\mathcal{F}_{lr}) + T(q)$. $\square$

Now we are ready to prove the security under the event E_3 , by constructing an OMDL challenger B_3 that uses the forger $\mathcal{F}_{lr}$, which is equivalent to the TS-SUF-4 forger $\mathcal{F}$.

Lemma 7. *There exists an OMDL adversary $\mathcal{B}_3$ issuing at most $2q_s + t$ queries to the CHALL oracle and q queries to random oracle such*

$$\Pr[E_3] \leq 8\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_3) + \text{negl}.$$

Moreover, $\mathcal{B}_3$ runs in time $T(\mathcal{B}_3) \leq \frac{24q \ln(24/\Pr[E_3])}{\Pr[E_3]} \cdot T(\mathcal{F}) + T(q)$ where ns is the number of signers, and $T(q)$ is the time required to perform exponentiations and group operations.

Proof. The proof setup is similar to that of Lemma 3 but considering the event

$$E_3 : \forall(\tilde{X}, M^*, \sigma^*) \wedge lr^* \neq \perp \wedge \neg(S_2(lr) = S_3(lr) = S_4(lr)).$$

As event E_2 has covered $S_2(lr) \neq S_3(lr)$, we only need to deal with the forgeries under $S_2(lr) = S_3(lr) \subset S_4(lr)$ in event E_3 and this is a subset of event E in Theorem 2. The simulation of algorithm $\mathcal{C}$ is the same as that of Lemma 3 but here, $\mathcal{C}$ runs $\mathcal{F}_{lr}$ instead of $\mathcal{F}$, and takes in $h_1, \dots, h_q$ only as the hash values. By Theorem 2, the TS-SUF-4 forger $\mathcal{F}$ and the leader request forger $\mathcal{F}_{lr}$ are equivalent. Unlike $\mathcal{F}$ which issues a PSIGNO query for lr^* to generate partial signatures before forging, $\mathcal{F}_{lr}$ outputs lr^* directly without issuing this specific query in its winning condition.

After receiving the output $(lr^* = \{M^*, \mathcal{S}, \text{PP} = \{pp_i = (R_i, S_i, u_i, V_i)\}_{\text{id}_i \in \mathcal{S}}\})$ from $\mathcal{F}_{lr}$, $\mathcal{C}$ returns $\perp$ if $\text{BadPPO} = \text{true}$, or E_3 does not occur. Else, denote $I = S_4(lr) \setminus S_3(lr)$ and assume $N = |I| = 1$, $\mathcal{C}$ knows $(I, \text{out}_{i^*}, \theta = \perp)$ with $I = \{i^*\}$ corresponds to a query $e_{i^*} = \text{H}_{\text{pp}}(\tilde{X}, X_{i^*}, R_{i^*}, S_{i^*}, V_{i^*})$ such that $pp_{i^*} \in lr^*$. If $1 < |I| < \text{ns}$, $\mathcal{C}$ selects a random index i^* from the $|I|$ indices. Then, $\mathcal{C}$ returns $(e_{i^*}, \text{out}_{i^*}) \leftarrow \mathcal{O}$.

Construct $\mathcal{B}_3$ from $\text{MIMFork}_{1,3}^{\mathcal{C}}$. We can construct the OMDL adversary $\mathcal{B}_3$ using $\text{MIMFork}_{1,3}^{\mathcal{C}}$ and the available DLOG oracle. To begin, $\mathcal{B}_3$ queries INIT and then queries CHALL oracle $2q_s + t$ times to generate the three lists of group elements $(\mathbf{A}, \mathbf{R}, \mathbf{S})$ as the input to run $\text{MIMFork}_{1,3}^{\mathcal{C}}$. All DLOG queries from $\text{MIMFork}_{1,3}^{\mathcal{C}}$ are forwarded by $\mathcal{B}_3$ to the DLOG oracle in the game $\mathbf{G}_{\mathbb{G}}^{\text{omdl}}$.

We first show how to compute the discrete logarithm of $A_0, \dots, A_{t-1}$. From $(\mathcal{O}, \mathcal{O}') \leftarrow \text{MIMFork}_{1,3}^{\mathcal{C}}$, $\mathcal{B}_3$ obtains $\{e_{i^*}, pp_{i^*} = (R_{i^*}, S_{i^*}, u_{i^*}, V_{i^*}) \leftarrow \text{out}_{i^*}\} \leftarrow \mathcal{O}$ and $\{e_{i^*,j}, pp_{i^*} = (R_{i^*}, S_{i^*}, u_{i^*,j}, V_{i^*})\}_{1 \leq j \leq 3} \leftarrow \mathcal{O}'$. This yields a solvable system of linear equations, allowing $\mathcal{B}_3$ to extract the discrete logarithms $f(\text{id}_{i^*}) = x_i \leftarrow \text{DL}(X_{i^*})$, $r_{i^*} \leftarrow \text{DL}(R_{i^*})$ and $s_{i^*} \leftarrow \text{DL}(S_{i^*})$. So, $\mathcal{B}_3$ knows $|\mathcal{CS}| + 1 = t$ pairs of $(\text{id}_i, f(\text{id}_i))$ and can interpolate the key generation polynomial $f(\cdot)$ to extract all $a_0 \leftarrow \text{DL}(A_0), \dots, a_{t-1} \leftarrow \text{DL}(A_{t-1})$. Since lr^* have not been queried to PSIGNO, for the OMDL challenges used as tokens $(R_i, S_i) \in lr^*$, $\mathcal{B}_3$ simply queries the DLOG oracle to solve them. For those $(R_i, S_i) \notin lr^*$ generated through

the PPO queries, they are either not queried, or queried once to PSIGNO before $\mathcal{F}_{lr}$ returns lr^* . $\mathcal{B}_3$ queries DLOG oracle to solve them as explained in Lemma 3.

Absorbing all strictly negligible terms such as $\Pr[\text{BadPPO}]$ and hash collision into a function negl , we have

$$\begin{aligned}
\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}_3) &\geq \text{acc}(\text{MIMFork}_{1,3}^{\mathcal{C}}) - \text{negl} \\
&\geq \frac{\text{Adv}_{\text{FROST2+}}^{\text{ts-lr}}(\mathcal{F}_{lr})}{8} - \text{negl} \\
&\geq \frac{\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F})}{8} - \text{negl} \quad // \text{by Theorem 2} \\
&\geq \frac{\Pr[E_3]}{8} - \text{negl},
\end{aligned}$$

and the running time is:

$$\begin{aligned}
T(\mathcal{B}_3) &\leq |I| \cdot T_{\max} \cdot T(\mathcal{F}_{lr}) + T(q) \\
&\leq \frac{N \cdot n \cdot 8 \cdot N \cdot q \ln(8 \cdot N \cdot n / \text{Adv}_{\text{FROST2+}}^{\text{ts-lr}}(\mathcal{F}_{lr}))}{\text{Adv}_{\text{FROST2+}}^{\text{ts-lr}}(\mathcal{F}_{lr})} \cdot T(\mathcal{F}_{lr}) + T(q) \\
&\leq \frac{24q \ln(24 / \text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}))}{\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F})} \cdot T(\mathcal{F}) + T(q) \quad // \text{by Theorem 2} \\
&\leq \frac{24q \ln(24 / \Pr[E_3])}{\Pr[E_3]} \cdot T(\mathcal{F}) + T(q).
\end{aligned}$$

□

4.1 Security with Distributed Key Generation

Our amendments on the pre-processing token generation and verification mechanism are orthogonal to key generation, allowing seamless integration with FROST2 when instantiated with secure distributed key generation (DKG) protocols such as PedPoP [22,6]. This follows because SPP/PS only use the existing key share $\text{st}_i.\text{sk}$ and token exponents (r, s) , and do not involve the DKG commitment/PoP messages. To ensure completeness, we demonstrate that our proposed scheme maintain TS-SUF-3 security in the ROM when it is instantiated in the DKG setting using PedPoP (in Appendix B).

Essentially, the oracle simulations are similar to that in Theorem 1, except for the key generation. We first show how to build an OMDL adversary $\mathcal{B}'_1$ for a forgery under event E_1 in the DKG setting. Intuitively, we construct the OMDL adversary $\mathcal{B}'_1$ using the mix forking technique [11] as follows. First, we define a wrapper $\mathcal{C}$ that is same as that in Lemma 3 but its $\widehat{\text{INIT}}$ simulates the PedPoP protocol. Then, an algorithm $\mathcal{D}$ runs $\text{Fork}^{\mathcal{C}}$ to extract the signing key $f(0)$. Next, we define $\mathcal{D}'$ which is syntactically a valid OMDL adversary but does not break the OMDL assumption. $\mathcal{D}'$ runs $\text{MIMFork}_{\text{CS},1}^{\mathcal{D}}$ and solves all the OMDL challenges. Finally, we present $\mathcal{B}'_1$, which uses $\mathcal{D}'$ as a helper to break the OMDL assumption.

We do not discuss the DKG-counterpart $\mathcal{B}'_2$ for the $\mathcal{B}_2$ in Lemma 4 which knows the signing key shares of every honest signer and it can run the PedPoP protocol honestly. Since $\mathcal{B}_1$ in Lemma 3 and $\mathcal{B}_3$ in Lemma 7 share the same game simulation strategy, the DKG-counterpart $\mathcal{B}'_3$ adopts the same key generation simulation as $\mathcal{B}'_1$ but perform a double forkings, that is, double layers of MIMFork, instead of a mix forking. Specifically, an algorithm $\mathcal{D}$ runs the first $\text{MIMFork}_{1,3}^C$ as in Lemma 7 to extract $f(0)$, followed by an algorithm $\mathcal{D}'$ which runs $\text{MIMFork}_{|CS|,1}^D$ to fork the $|CS| < t$ PoPs generated by the forger in PedPoP. Finally, $\mathcal{B}'_3$ runs $\mathcal{D}'$ and halts $\mathcal{C}$ right after PedPoP is completed to solve an extra OMDL challenges and break the OMDL assumption — exactly the same proving steps of $\mathcal{B}'_1$ as shown below.

Theorem 3. *Let $\mathcal{F}$ be a TS-SUF-4 forger against FROST2+ with PedPoP making at most q queries to the random oracle. Suppose the success probability of $\mathcal{F}$ is bounded by $\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \Pr[E_1] + \Pr[E_2] + \Pr[E_3]$. Then, there exist three OMDL adversaries $\mathcal{B}'_1, \mathcal{B}'_2$ and $\mathcal{B}'_3$ such that:*

$$\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \sqrt{8q \cdot \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}'_1)} + 8\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}'_2) + 64\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}'_3) + \text{negl}$$

where $\mathcal{B}'_1$ and $\mathcal{B}'_3$ makes at most $2q_s + t$ queries to the CHALL oracle, and $\mathcal{B}'_2$ makes at most $2q_s$ queries. Moreover, the running times of $\mathcal{B}'_1, \mathcal{B}'_2$ and $\mathcal{B}'_3$ are bounded above by

$$\begin{aligned} T(\mathcal{B}'_1) &\leq \frac{128t^2q^2 \ln(64tq / \Pr[E_1]^2)}{\Pr[E_1]^2} \cdot T(\mathcal{F}) + T(q), \\ T(\mathcal{B}'_2) &\leq \frac{16ns^2q \ln(16ns / \Pr[E_2])}{\Pr[E_2]} \cdot T(\mathcal{F}) + T(q), \text{ and} \\ T(\mathcal{B}'_3) &\leq \frac{12288t^2q^2 \ln(512t / \Pr[E_3]) \ln(24 / \Pr[E_3])}{\Pr[E_3]^2} \cdot T(\mathcal{F}) + T(q), \end{aligned}$$

respectively, where $t \leq ns$, and $T(q)$ denotes the time required for the relevant group operations and exponentiations.

Proof. We show how to construct the $\mathcal{B}'_1$ OMDL adversary before showing the $\mathcal{B}'_3$ OMDL adversary which shares the same proving strategy.

The $\mathcal{B}'_1$ Adversary. We define an algorithm $\mathcal{D}$ which can be viewed as the DKG-counterpart of $\mathcal{B}_1$ in Lemma 3. It constructs a wrapper $\mathcal{C}$ and simulates the oracles in a similar way. However, $\mathcal{C}$ now defines $f_i(Z)$ for every honest signer instead of querying the OMDL oracles to implicitly define $f(Z)$. $\mathcal{C}$ also defines an additional $\text{RO}[\text{H}_{\text{kg}}]$ oracle to handle the hash queries incurred by PedPoP:

$\text{RO}[\text{H}_{\text{kg}}](x)$: If $\text{H}_{\text{kg}}(x) \neq \perp$, return $\text{H}_{\text{kg}}(x)$. Else, $\mathcal{C}$ sets $\text{ctr}_{pp} \leftarrow \text{ctr}_{pp} + 1$ and $\text{H}_{\text{kg}}(x) \leftarrow h_{\text{ctr}_{pp}}$ then returns $\text{H}_{\text{kg}}(x)$.

Particularly, in $\widetilde{\text{INIT}}(CS)$, let honest signers be the set $HS = \{\text{id}_i, \dots, \text{id}_{n_s}\} \setminus CS$ and $k = |CS| < t$, $\mathcal{C}$ queries $A_0 = g^{a_0}, \{A_\ell = g^{a_\ell}\}_{\ell=1}^k \leftarrow \text{CHALL}$ and samples $\{a_\ell\}_{\ell=k+1}^{t-1} \xleftarrow{\$} \mathbb{Z}_p^*$. $\mathcal{C}$ selects $\{y_i\}_{\text{id}_i \in HS} \xleftarrow{\$} \mathbb{Z}_p^*$ to implicitly define the random polynomials $f_i(Z) = y_i(\sum_{\ell=0}^{t-1} a_\ell Z^\ell)$ for every honest signer $\text{id}_i \in HS$. Firstly, for every $\text{id}_i \in HS$, $\mathcal{C}$ increments ctr_{pp} and chooses $\bar{z}_i \xleftarrow{\$} \mathbb{Z}_p^*$ to compute $\bar{R}_i = g^{\bar{z}_i} / A_0^{y_i h_{\text{ctr}_{pp}}}$ and set $H_{\text{kg}}(A_{i,0} = A_0^{y_i}, A_{i,0}, \bar{R}_i) \leftarrow h_{\text{ctr}_{pp}}$. $\mathcal{C}$ broadcasts $((\bar{R}_i, \bar{z}_i), \mathbf{C} = (A_{i,0} = A_0^{y_i}, \dots, A_{i,t-1} = A_{t-1}^{y_i}))$ as the proof of possession (PoP). Secondly, $\mathcal{C}$ queries $f'(\text{id}_j) \leftarrow \text{DLOG}(A_0 \prod_{\ell=1}^{t-1} A_\ell^{\text{id}_j^\ell})$ and sends $x_{i,j} = y_i f'(\text{id}_j)$ from every honest signer $\text{id}_i \in HS$ to every corrupted signer $\text{id}_j \in CS$. In return, $\mathcal{C}$ receives $x_{j,i} = f_j(\text{id}_i)$ from $\text{id}_j \in CS$ for every $\text{id}_i \in HS$. So far, $\mathcal{C}$ issued $|CS| + 1$ CHALL queries and $|CS|$ DLOG queries.

As $\mathcal{C}$ simulates the DKG perfectly, all PoP broadcast by HS are verified and every corrupted signer $\text{id}_j \in CS$ obtains their signing key share $x_j = f(\text{id}_j) = \sum_{i=1}^{n_s} f_i(\text{id}_j) = \sum_{i=1}^{n_s} x_{i,j}$ as expected, and the joint public key is $\tilde{X} = g^{a_0 \sum_{\text{id}_i \in HS} y_i + \sum_{\text{id}_j \in CS} a_{j,0}} = \prod_{\text{id}_i \in HS} A_{i,0} \prod_{\text{id}_j \in CS} A_{j,0}$. Furthermore, from $\mathcal{F}$'s view, each signer's state st is indistinguishable from that generated in the CKG setting, so the oracle simulations by $\mathcal{D}$ after the key generation stage are identical to that by $\mathcal{B}_1$ in Lemma 3.

Therefore, from $(i_{\text{sig}}, \text{Out}) \leftarrow \text{Fork}^{\mathcal{C}}$, $\mathcal{D}$ can extract the signing key $f(0) = a_0 \sum_{\text{id}_i \in HS} y_i + \sum_{\text{id}_j \in CS} a_{j,0}$. However, $\mathcal{D}$ does not proceed to interpolate $f(\cdot)$ because it is not helpful in solving its OMDL challenges. Specifically, the discrete logarithms required here are no longer the $f(\cdot)$ coefficients (the OMDL challenges of $\mathcal{B}_1$ in Lemma 3), but the values $\{a_{j,0} = f_j(0)\}_{\text{id}_j \in CS}$ which are not known to $\mathcal{D}$.

Construct helper $\mathcal{D}'$ from $\text{MIMFork}_{|CS|,1}^{\mathcal{D}}$. Denote the output of $\mathcal{D}$ as $(I, \{\text{out}_i\}_{i \in I}, \theta)$ where out_i is the internal state of $\mathcal{D}$ up to the i -th random oracle response such that $I = (i_1, \dots, i_{|CS|})$ are the set of random oracle indices for $h_i \leftarrow H_{\text{kg}}(\text{id}_i, A_{i,0}, A_{i,0}, \bar{R}_i)$ during $\widetilde{\text{INIT}}$ and $\theta = f(0)$. We now construct a helper algorithm $\mathcal{D}'$ which is a syntactically valid OMDL adversary and it runs $\text{MIMFork}_{|CS|,1}^{\mathcal{D}}$. $\mathcal{D}'$ generates uniformly random $h_1, \dots, h_{2q} \in \mathbb{Z}_p^*$ and queries the $2q_s + t$ challenges $(\mathbf{A}, \mathbf{R}, \mathbf{S}) \leftarrow \text{CHALL}$ to initiate $\text{MIMFork}_{|CS|,1}^{\mathcal{D}}$. All DLOG queries from $\mathcal{D}$ are relayed to $\mathcal{D}'$.

Therefore, from $(\mathcal{O}, \mathcal{O}') \leftarrow \text{MIMFork}_{|CS|,1}^{\mathcal{D}}$, $\mathcal{D}'$ obtains $f(0)$ from the first run of $\mathcal{D}$, and extracts the $\{f_j(0) = a_{j,0}\}_{\text{id}_j \in CS}$ from corrupted signers' PoPs $((\bar{R}_j, \bar{z}_j = \bar{r}_j + h_j a_{j,0}) \leftarrow \mathcal{O}, (\bar{R}_j, \bar{z}'_j = \bar{r}_j + h'_j a_{j,0}) \leftarrow \mathcal{O}')$. Now, $\mathcal{D}'$ can compute $f'(0) = a_0 = \left(f(0) - \sum_{\text{id}_j \in CS} f_j(0)\right) / \sum_{\text{id}_i \in HS} y_i$. With the knowledge of $(f'(0), \{f'(\text{id}_j)\}_{\text{id}_j \in CS}, \{a_\ell\}_{\ell=|CS|+1}^{t-1})$, $\mathcal{D}'$ can recover $f'(\cdot)$ and then $\{a_\ell \leftarrow \text{DL}_{\mathbb{G},g}(A_\ell)\}_{\ell=1}^{|CS|}$ through Lagrange interpolation. Hence, $\mathcal{D}'$ solved all $|CS| + 1$ challenges $\{a_\ell \leftarrow \text{DL}(A_\ell)\}_{\ell=0}^{|CS|}$ used during $\widetilde{\text{INIT}}$ and it can also solve the $2q_s$ challenges $\{r_\ell \leftarrow \text{DL}(R_\ell), s_\ell \leftarrow \text{DL}(S_\ell)\}_{\ell=1}^{q_s}$ used in PPO queries as $\mathcal{B}_1$ done in Lemma 3. Since the BadPPO and hash collision happen with a negligible

probability, the success probability of $\mathcal{D}'$ is:

$$\text{acc}(\mathcal{D}') \geq \text{acc}(\text{MIMFork}_{|CS|,1}^{\mathcal{D}}) - \text{negl} \geq \text{acc}(\text{Fork}^{\mathcal{C}})/8 - \text{negl} \geq \Pr[E_1]^2/8q - \text{negl}$$

However, rewinding $\mathcal{D}$ for at most $|I| \cdot T_{\max}$ times making $\mathcal{D}'$ issued more than $2q_s$ DLOG queries to answer PSIGNO queries and so $\mathcal{D}'$ does not solve OMDL.

Construct $\mathcal{B}'_1$ from $\mathcal{D}'$. We now construct an OMDL adversary $\mathcal{B}'_1$ which is defined the same as $\mathcal{D}'$ except that $\mathcal{B}'_1$ aborts any executions of $\mathcal{C}$ after it obtains all PoPs $(\bar{R}_j, \bar{z}_j)$ which $\mathcal{F}$ sent during the $\widetilde{\text{INIT}}$ query. This makes sure $\mathcal{D}'$'s internal state recorded the required $(\theta, \{(\bar{R}_j, \bar{z}_j, \bar{z}'_j)\}_{\text{id}_j \in CS})$, yet not issuing any additional DLOG query during the rewinds as $\mathcal{D}'$ suffered from. Therefore, $\mathcal{B}'_1$ has the same success probability as $\mathcal{D}'$ and it runs in time:

$$\begin{aligned} T(\mathcal{B}'_1) &\leq T(\mathcal{D}') \leq |I| \cdot T_{\max} \cdot T(\mathcal{D}) + T(q) \\ &\leq t \cdot T_{\max} \cdot 2T(\mathcal{F}) + T(q) \\ &\leq \frac{8t^2q \ln(8t/\text{acc}(\mathcal{D}'))}{\text{acc}(\mathcal{D}')} \cdot 2T(\mathcal{F}) + T(q) \\ &\leq \frac{128t^2q^2 \ln(64tq/\Pr[E_1]^2)}{\Pr[E_1]^2} \cdot T(\mathcal{F}) + T(q) \end{aligned}$$

such that $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}'_1) \geq \Pr[E_1]^2/8q - \text{negl}$.

The $\mathcal{B}'_3$ Adversary. As explained previously, $\mathcal{B}'_3$ which handles forgery under event E_3 follows the same proving steps of $\mathcal{B}'_1$ although $\mathcal{B}'_3$ executes double MIMFork but not performing a mix forking. Therefore, for $\mathcal{B}'_3$, the success probability of $\mathcal{D}'$ is:

$$\text{acc}(\mathcal{D}') \geq \text{acc}(\text{MIMFork}_{|CS|,1}^{\mathcal{D}}) - \text{negl} \geq \text{acc}(\text{MIMFork}_{1,3}^{\mathcal{C}})/8 - \text{negl} \geq \Pr[E_3]/64 - \text{negl}$$

and $\mathcal{B}'_3$ runs in time:

$$\begin{aligned} T(\mathcal{B}'_3) &\leq T(\mathcal{D}') \leq |I| \cdot T_{\max} \cdot T(\mathcal{D}) + T(q) \\ &\leq t \cdot T_{\max} \cdot \frac{24q \ln(24/\Pr[E_3])}{\Pr[E_3]} \cdot T(\mathcal{F}) + T(q) \\ &\leq \frac{8t^2q \ln(8t/\text{acc}(\mathcal{D}'))}{\text{acc}(\mathcal{D}')} \cdot \frac{24q \ln(24/\Pr[E_3])}{\Pr[E_3]} T(\mathcal{F}) + T(q) \\ &\leq \frac{512t^2q \ln(512t/\Pr[E_3])}{\Pr[E_3]} \cdot \frac{24q \ln(24/\Pr[E_3])}{\Pr[E_3]} T(\mathcal{F}) + T(q) \\ &\leq \frac{12288t^2q^2 \ln(512t/\Pr[E_3]) \ln(24/\Pr[E_3])}{\Pr[E_3]^2} \cdot T(\mathcal{F}) + T(q) \end{aligned}$$

such that $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}'_3) \geq \Pr[E_3]/64 - \text{negl}$. $\square$

5 Performance Evaluations

We benchmark the proposed FROST2+ against ZCash’s FROST implementation (GitHub commit 3ffc19d) [29] on a laptop with an Intel i9-14900HX (2.20 GHz), 32 GB RAM, running Windows 11 Home Single Edition. To support our (and FROST2’s) CompPar algorithm, which differs from the original FROST design, we modified the functions `binding_factor_preimages` and `compute_group_commitment` in `frost-core/src/lib.rs`.

We benchmark the related schemes by measuring only their signing throughput. Variants such as ROAST [26] and Olaf [11] build on FROST2 and add robustness mechanisms. When those mechanisms are not invoked, their signing cost is essentially that of FROST2. Similarly, ATS[FROST,DS] is FROST plus an additional (Schnorr) signature per pre-processing token, so it is dominated by the measured FROST cost. We therefore benchmark FROST, FROST2, ms-FROST2 and FROST2+, which represent the main computational differences in Table 2, using the Ed25519 curve across various predefined $(t = 0.7n, n)$ parameter sets. For each configuration, we executed 100 rounds and recorded the average time per signature generation. Each timing experiment was conducted twice: once with pre-computations enabled and once with them disabled. When pre-computations were enabled, the timings for token verification and the pre-processing round were excluded from the signature generation time; when disabled, they were included. The full numerical results are provided in Table 3 (Appendix C), and Figure 3 visualizes the signatures-per-second for the four schemes. We also present a comparison of the algorithmic complexity which focuses on scalar multiplications, and security of our schemes against FROST and its two-round variants in Table 2. This analysis highlights the core computational differences in scalar operations, with pre-processing contributions denoted in parentheses.

When pre-computations are enabled, FROST2+ achieves throughput nearly identical (within -2.3% to 1.8%) to FROST2 across all group sizes, demonstrating that its additional features come with negligible overhead. Besides, FROST2+ is always $64.7\% = \frac{3333-2024}{2024} \times 100$ (at $n = 50$) to $79\% = \frac{1267-708}{708} \times 100$ (at $n = 150$) faster than FROST with pre-computations, confirming the better scalability of the underlying algorithms in FROST2 and FROST2+. Specifically, at $n = 200$ signers, FROST2+ reaches 913 signatures/sec, marginally surpassing FROST2 (897 sig/s) and outperforming FROST (544 sig/s). ms-FROST shows a similar performance to FROST, suggesting that the hashing operation is also an important factor for signing speed.

When pre-computations are disabled, FROST2+ exhibits a noticeable performance degradation compared to FROST2 (e.g., 2160 vs. 3012 sig/s at $n = 50$). This suggests that the optimizations in FROST2+ rely heavily on the pre-computations to achieve competitive throughput, and without it the additional operations from authenticating tokens impose a measurable cost. Nonetheless, even without pre-computations, FROST2+ outperforms FROST for all tested n , demonstrating the efficiency gained from the lighter-weight CompPar algorithm.

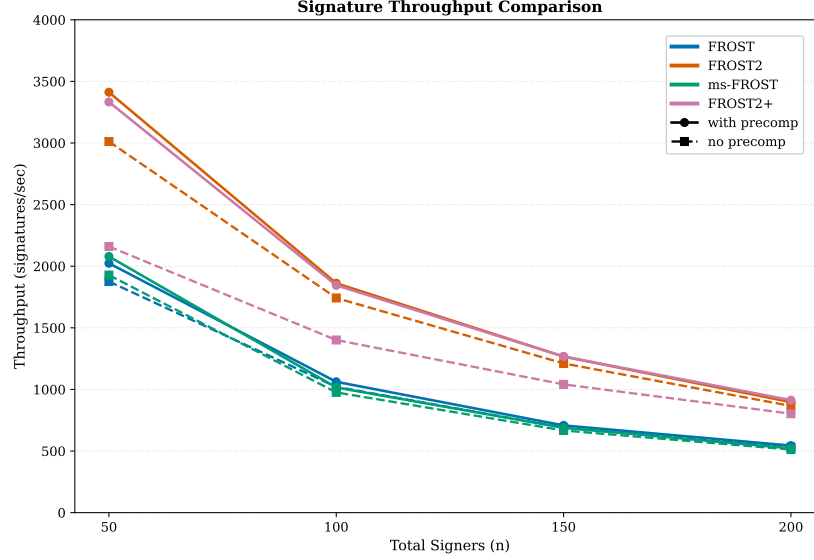


Fig. 3. Signatures generated per seconds for FROST, FROST2, ms-FROST, FROST2+.

Table 2. Comparison of complexity and security for the proposed FROST+ against FROST and its variants with two signing rounds.

Scheme	Sign			Agg		Vf	Security				
	Round	$ pp $	$smul$	$\mathbb{G}$	$\mathbb{Z}_p$		$smul$	Notion	Model	Corrupt	Kg
FROST [22]	2	2	$(2) + t$	2	1	t	2	TS-UF-1	ROM	Static	D
FROST [6]	2	2	$(2) + t$	3	1	0	2	TS-SUF-3	ROM	Static	C
FROST** [14,17]	2	2	$(2) + t$	2	1	t	2	adp-TS-UF	AGM	Adaptive	C
ATS[TS, DS]* [6]	2	4	$(3) + 3t$	4	2	0	2	TS-SUF-4	ROM	Static	C
FROST2 [15]	2	2	$(2)+1$	2	1	1	2	TS-UF-1	ROM	Static	D
FROST2 [6]	2	2	$(2)+1$	3	1	0	2	TS-SUF-2	ROM	Static	C
ROAST [26]	2	2	$(2)+1$	2	1	1	2	TS-SUF-2	ROM	Static	C
Olaf [11]	2	2	$(2)+1$	2	1	1	2	TS-SUF-2	ROM	Static	D
Arctic [23]	2	1	$(1) + t^2 + t$	1	1	t	2	UF	ROM	Static	C,D
ms-FROST [2]	2	2	$(2)+1$	3	1	0	2	adp-TS-UF-4	AGM	Adaptive	C
FaFROST [4]	2	2	$(2)+1$	3	1	0	2	adp-TS-UF	AGM	Adaptive	C
FROST-Mask [10]	2	2	$(2)+1$	3	1	1	2	adp-UF	ROM	Adaptive	C
FROST2** [17]	2	2	$(2) + 1$	3	1	0	2	adp-TS-UF	AGM	Adaptive	C
FROST3** [17]	2	2	$(2) + 1$	2	1	1	2	adp-TS-UF	AGM	Adaptive	C
FROST2+	2	4	$(3) + 4t + 1$	4	2	0	2	TS-SUF-4	ROM	Static	C,D
FROST2+[§]	2	4	$(3 + 4t) + 1$	4	2	0	2	TS-SUF-4	ROM	Static	C,D

* Assume DS is the Schnorr signature scheme. [§]with pre-computations.

** Maximum of $t/2$ adaptive corruptions if in ROM.

Note: $smul$: scalar multiplication, $\mathbb{G}, \mathbb{Z}_p$: numbers of group and field element sent per signer, Kg: key generation, C: centralized, D: decentralized, t : threshold, (num) : op. in pre-proc. round

6 Conclusion

We presented an enhanced FROST2 scheme, namely, FROST2+ with proven TS-SUF-4 security under OMDL assumption in the ROM with static corruption and (de)centralized key generation settings. FROST2+ is the only ROM-based TS-SUF-4 scheme to date apart from Bellare et al.’s ATS[FROST,DS] where the primitive FROST used there is at least 64% slower than FROST2+ for a signing group with threshold $t = 0.7n$.

References

1. Arch Network: Arch’s signature scheme model (frost + roast). <https://docs.arch.network/learn/fundamentals/arch-signature-scheme-model-frost-roast> (2024), documentation page, last updated 2024; accessed Nov 18, 2025
2. Bacho, R., Chen, Y., Loss, J., Tessaro, S., Zhu, C.: Adaptively secure partially non-interactive threshold schnorr signatures in the AGM. Cryptology ePrint Archive, Paper 2025/1953 (2025), <https://eprint.iacr.org/2025/1953>
3. Bacho, R., Das, S., Loss, J., Ren, L.: Adaptively secure three-round threshold schnorr signatures from ddh. In: Advances in Cryptology – CRYPTO 2025: 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VI. p. 390–422. Springer-Verlag, Berlin, Heidelberg (2025). https://doi.org/10.1007/978-3-032-01887-8_13, https://doi.org/10.1007/978-3-032-01887-8_13
4. Baecker, R., Gerhart, P., Calsi, D.L., Russo, L., Schröder, D., Yerukhimovich, A.: Fully adaptive FROST in the algebraic group model from falsifiable assumptions. Cryptology ePrint Archive, Paper 2025/1950 (2025), <https://eprint.iacr.org/2025/1950>
5. Bagherzandi, A., Cheon, J.H., Jarecki, S.: Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In: Proceedings of the 15th ACM Conference on Computer and Communications Security. p. 449–458. CCS ’08, Association for Computing Machinery, New York, NY, USA (2008). <https://doi.org/10.1145/1455770.1455827>, <https://doi.org/10.1145/1455770.1455827>
6. Bellare, M., Crites, E., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Better than advertised security for non-interactive threshold signatures. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022. pp. 517–550. Springer Nature Switzerland, Cham (2022)
7. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. p. 390–399. CCS ’06, Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1180405.1180453>, <https://doi.org/10.1145/1180405.1180453>
8. Bellare, M., Tessaro, S., Zhu, C.: Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Paper 2022/833 (2022), <https://eprint.iacr.org/2022/833>
9. Brandão, L.T.A.N., Peralta, R.: NIST First Call for Multi-Party Threshold Schemes (NISTIR 8214C, draft). NIST Interagency Report (NISTIR) – Draft 8214C, National Institute of Standards and Technology (NIST) (2023), <https://csrc.nist.gov/publications/detail/nistir/8214c/draft>

10. Chen, Y.: Round-efficient adaptively secure threshold signatures with rewinding. *IACR Communications in Cryptology* **2**(2) (2025). <https://doi.org/10.62056/ah893z10k>
11. Chu, H., Gerhart, P., Ruffing, T., Schröder, D.: Practical schnorr threshold signatures without the algebraic group model. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023*. pp. 743–773. Springer Nature Switzerland, Cham (2023)
12. Connolly, D., Komlo, C., Goldberg, I., Wood, C.A.: Rfc 9591 the flexible round-optimized schnorr threshold (frost) protocol for two-round schnorr signatures (2024)
13. Cremers, C., Peltonen, A., Zhao, M.: An extended hierarchy of security notions for threshold signature schemes and automated analysis of protocols that use them. *Cryptology ePrint Archive*, Paper 2024/1920 (2024), <https://eprint.iacr.org/2024/1920>
14. Crites, E., Katz, J., Komlo, C., Tessaro, S., Zhu, C.: On the adaptive security of frost. In: Tauman Kalai, Y., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025*. pp. 480–511. Springer Nature Switzerland, Cham (2025)
15. Crites, E., Komlo, C., Maller, M.: How to prove schnorr assuming schnorr: Security of multi- and threshold signatures. *Cryptology ePrint Archive*, Paper 2021/1375 (2021), <https://eprint.iacr.org/2021/1375>
16. Crites, E., Komlo, C., Maller, M.: Fully adaptive schnorr threshold signatures. In: *Advances in Cryptology – CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part I*. p. 678–709. Springer-Verlag, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-031-38557-5_22, https://doi.org/10.1007/978-3-031-38557-5_22
17. Crites, E., Komlo, C., Maller, M.: On the adaptive security of key-unique threshold signatures. *Cryptology ePrint Archive*, Paper 2025/943 (2025), <https://eprint.iacr.org/2025/943>
18. Crites, E., Stewart, A.: A plausible attack on the adaptive security of threshold schnorr signatures. In: Tauman Kalai, Y., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025*. pp. 457–479. Springer Nature Switzerland, Cham (2025)
19. Gennaro, R., Leigh, D., Sundaram, R., Yezauris, W.: Batching schnorr identification scheme with applications to privacy-preserving authorization and low-bandwidth communication devices. In: Lee, P.J. (ed.) *Advances in Cryptology – ASIACRYPT 2004 (Lecture Notes in Computer Science, vol. 3329)*. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30539-2_20
20. Harman, S., Shishmarev, M., Nash, T., Lennon, D., Janse van Vuuren, S.: Chainflip protocol whitepaper. Tech. rep., Auxilio Ventures (2022), <https://www.auxilioventures.com/wp-content/uploads/2022/08/Chainflip-Whitepaper-May-2020.pdf>
21. Kiltz, E., Masny, D., Pan, J.: Optimal security proofs for signatures from identification schemes. In: Robshaw, M., Katz, J. (eds.) *Advances in Cryptology – CRYPTO 2016*. pp. 33–61. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
22. Komlo, C., Goldberg, I.: Frost: Flexible round-optimized schnorr threshold signatures. In: Dunkelman, O., Jacobson, Jr., M.J., O’Flynn, C. (eds.) *Selected Areas in Cryptography*. pp. 34–65. Springer International Publishing, Cham (2021)
23. Komlo, C., Goldberg, I.: Arctic: Lightweight and stateless threshold schnorr signatures. In: Jager, T., Pan, J. (eds.) *Public-Key Cryptography – PKC 2025*. pp. 234–267. Springer Nature Switzerland, Cham (2025)

24. Mccorry, P., Bakshi, S., Dembo, A., Miller, A.: sbtc: Design of a trustless two-way peg for bitcoin. Tech. rep., Trustless Computing Foundation (2023), <https://sbtc.tech/sbtc.pdf>
25. Niot, G., Reichle, M., Takemure, K.: Adaptively-secure three-round threshold schnorr from DL. Cryptology ePrint Archive, Paper 2025/1941 (2025), <https://eprint.iacr.org/2025/1941>
26. Ruffing, T., Ronge, V., Jin, E., Schneider-Bensch, J., Schröder, D.: Roast: Robust asynchronous schnorr threshold signatures. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 2551–2564. CCS '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3548606.3560583>, <https://doi.org/10.1145/3548606.3560583>
27. Safe Research: safe-frost. <https://github.com/safe-research/safe-frost> (2024), gitHub repository; accessed Nov 18, 2025
28. Schnorr, C.P.: Efficient signature generation by smart cards. Journal of Cryptology 4, 161–174 (1991). <https://doi.org/10.1007/BF00196725>
29. Zcash Foundation: Speeding up frost with multi-scalar multiplication, <https://zfnf.org/speeding-up-frost-with-multi-scalar-multiplication/>

A Attacks on FROST2

A.1 Types of Forgeries

Referring to the security model in Section 2.4, we recall the conditions under which a (strong) forgery can be achieved in a threshold signature scheme that is also an echo scheme [8,6] (e.g., FROST2) in Figure 4.

A.2 TS-UF-3 Attack

In Bellare et al.’s work [8,6], they provide a security proof of TS-SUF-2 and also a TS-UF-3 attack for FROST2. This indicates that the best possible security level FROST2 can achieve is only TS-SUF-2. We recall Bellare et al.’s TS-UF-3 attack in the setting below:

- Total signers are $\{P_1, P_2, P_3, P_4\}$.
- Signers $\{P_3, P_4\}$ corrupted.
- A signing request on signer set $lr.\mathcal{S} = \{1, 2, 3\}$ for a message $lr.msg = M$.
- The pre-processing token $(R_1, S_1), (R_2, S_2)$ are known to the corrupted signers before they generate theirs.
- First signer’s partial signature $z_1 = r_1 + ds_1 + c\Delta_{1,\mathcal{S}}(0)x_1$ can be obtained.
- Expected combined signature is $z = d\log(\tilde{R}) + cx = \sum_{i \in \mathcal{S}} r_i + ds_i + c\Delta_{i,\mathcal{S}}(0)x_i$

The attack procedure is as follows:

1. P_3 generates its pre-processing token as

$$R_3 = R_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} - 1} R_2^{-1}, S_3 = S_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} - 1} S_2^{-1}.$$

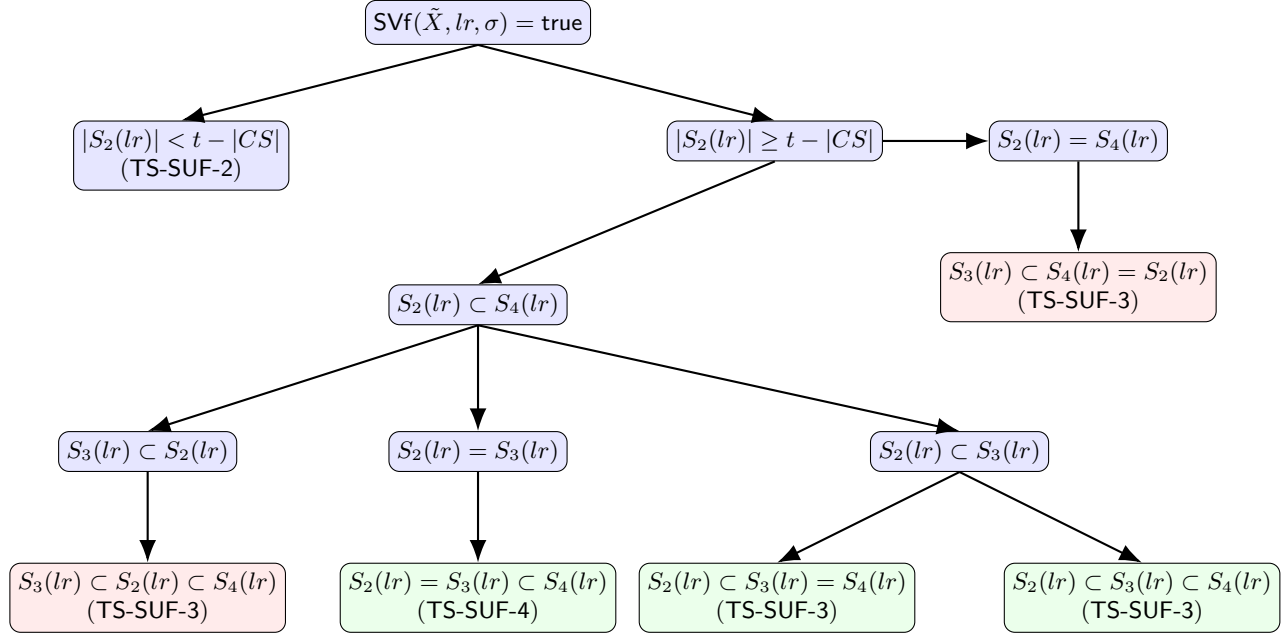


Fig. 4. Decision tree for queries comparisons and forgery outcomes. marks the algorithmically prohibited forgery types (i.e., $\text{st.mapPP}(pp_i) \neq \perp$ enforced in PS) while marks the possible forgery types.

where $\mathcal{S}' = \{1, 3, 4\}$. The resulting group pre-processing token will be:

$$\tilde{R} = R_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}} S_1^{d \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}} = \prod_{i \in \mathcal{S}} R_i S_i^d$$

where $\text{dlog}(\tilde{R}) = r_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + ds_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}$ and $d = H_{\text{non}}(\tilde{X}, lr)$.

2. P_3 computes $z'_1 = r_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + ds_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + c\Delta_{1,\mathcal{S}'(0)}x_1 = \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}z_1$
3. P_3 and P_4 forge a valid combined signature for M under $lr.\mathcal{S}$ as

$$z' = \text{dlog}(\tilde{R}) + cx = z'_1 + c\Delta_{3,\mathcal{S}'(0)}x_3 + c\Delta_{4,\mathcal{S}'(0)}x_4.$$

This is a valid TS-UF-3 attack because $S_2 = \{1\}$ but unequal to $S_3 = \{1, 2\}$, i.e., $S_2(lr) \subset S_3(lr) = S_4(lr)$ in Figure 4. We note that this attack does not work on our schemes because the attacker's pre-processing token (R_3, S_3) is a combination of the honest signers' pre-processing token whose discrete logarithms are not known. Consequently, a valid authentication transcript (u, V) for (R_3, S_3) cannot be generated and the honest signers abort during PSIGNO.

A.3 TS-UF-4 Attack

In Bellare et al.'s work [8,6], they demonstrate a TS-UF-4 attack for FROST but not FROST2. Here, we mount the attack on FROST2 as follows:

- Total signers are $\{P_5, P_{10}, P_{11}, P_{15}, P_{20}\}$.
- Signers $\{P_5, P_{10}\}$ corrupted.
- A signing request on signer set $lr.S = \{11, 15, 20\}$ for a message $lr.msg = M$.
- The pre-processing token $lr.PP = \{(R_{11}, S_{11}), (R_{15} = g^{r_{15}}, S_{15} = g^{s_{15}}), (R_{20} = g^{r_{20}}, S_{20} = g^{s_{20}})\}$ and $r_{15}, s_{15}, r_{20}, s_{20} \in \mathbb{Z}_p^*$ are known to the corrupted signers before they generate theirs.
- First signer's partial signature $z_{11} = r_{11} + d_{11}s_{11} + c\Delta_{11,S}(0)x_{11}$ can be obtained.
- Expected combined signature is $z = \text{dlog}(\tilde{R}) + cx = \sum_{i \in S} r_i + ds_i + c\Delta_{i,S}(0)x_i$

The attack procedure is as follows:

1. Let $S' = \{5, 10, 11\}$, the Lagrange coefficients are the same for:

$$\Delta_{11,lr.S} = \frac{15 \cdot 20}{(15 - 11)(20 - 11)} = \frac{25}{3} = \frac{5 \cdot 10}{(5 - 11)(10 - 11)} = \Delta_{11,S'}.$$

The resulting group pre-processing token will be:

$$\tilde{R} = R_{11}S_{11}^d g^{r_{15} + ds_{15} + r_{20} + ds_{20}} = \prod_{i \in S} R_i S_i^d$$

where $\text{dlog}(\tilde{R}) = r_{11} + ds_{11} + r_{15} + ds_{15} + r_{20} + ds_{20}$ and $d = H_{\text{non}}(\tilde{X}, lr)$.

2. P_5 and P_{10} forge a valid combined signature for M under $lr.S$ as

$$z' = \text{dlog}(\tilde{R}) + cx = z_{11} + r_{15} + ds_{15} + r_{20} + ds_{20} + c\Delta_{5,S'}(0)x_5 + c\Delta_{10,S'}(0)x_{10}.$$

This is a valid TS-UF-4 attack because $S_2 = \{11\} = S_3$ but unequal to $S_4 = \{11, 15, 20\}$, i.e., $S_2(lr) = S_3(lr) \subset S_4(lr)$ in Figure 4. We note that this attack does not work on our schemes for the similar reason in Section A.2. Although the attacker knows the discrete logarithms for the pre-processing tokens $(R_{15}, S_{15}), (R_{20}, S_{20})$, it does not know the honest signers' signing key shares which are required to generate the authentication transcripts $(u_{15}, V_{15}), (u_{20}, V_{20})$ for the tokens. Therefore, the honest signer will abort during partial signing because of invalid $(u_{15}, V_{15}), (u_{20}, V_{20})$.

B PedPoP

We recall PedPoP key generation [22,6] in Figure 5.

C Detailed Benchmark Results

The detailed benchmark data is in Table 3.

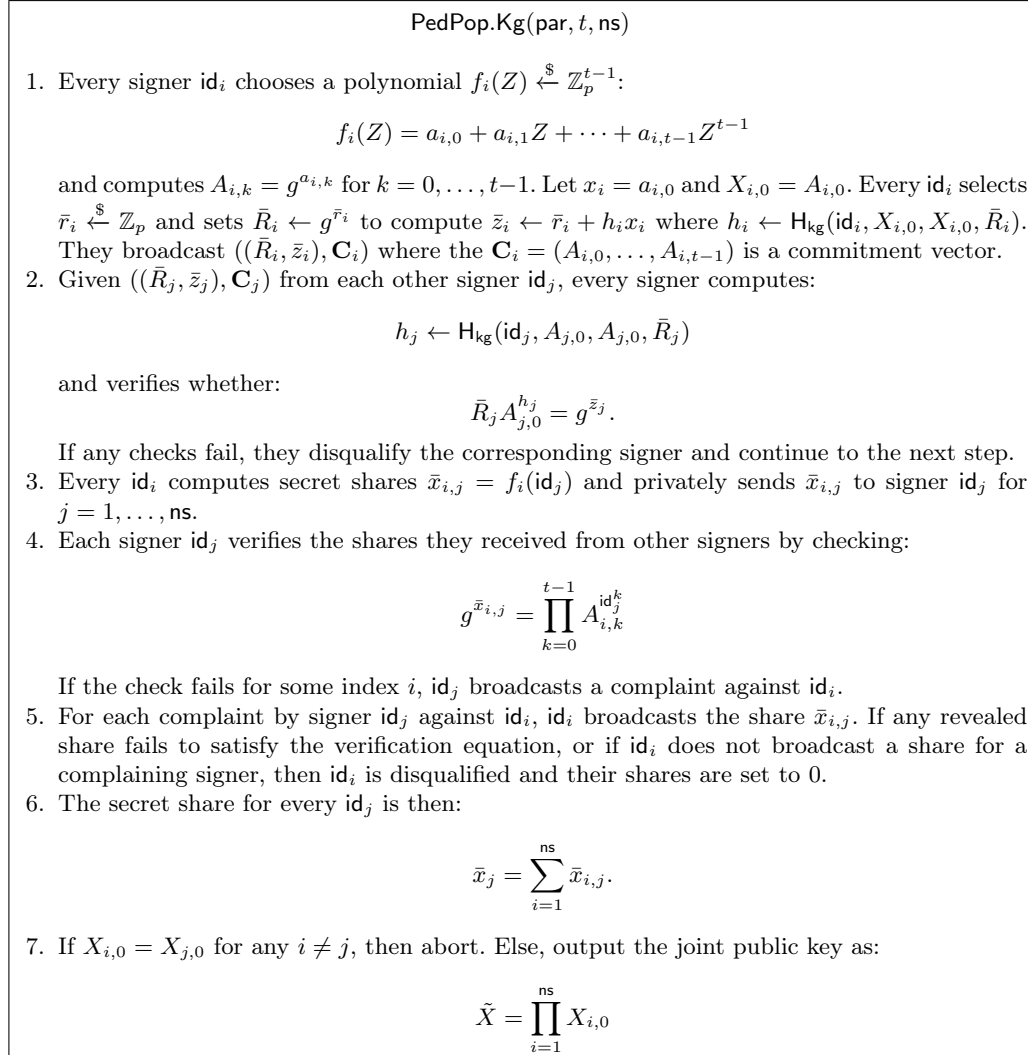


Fig. 5. The PedPoP algorithm [22,6].

Table 3. Performance comparison of threshold signature schemes (excluding DKG setup). (t, n) denotes the threshold and number of signers.

Scheme	Pre-comp	(t, n)	Signature Gen. (ms)	Throughput (sig/sec)	Security
FROST	✓	(35, 50)	0.494	2,024	TS-SUF-3
		(70, 100)	0.941	1,062	
		(105, 150)	1.412	708	
		(140, 200)	1.838	544	
	✗	(35, 50)	0.533	1,876	
		(70, 100)	0.979	1,021	
		(105, 150)	1.449	690	
		(140, 200)	1.879	532	
FROST2	✓	(35, 50)	0.293	3,412	TS-SUF-2
		(70, 100)	0.537	1,862	
		(105, 150)	0.789	1,267	
		(140, 200)	1.115	897	
	✗	(35, 50)	0.332	3,012	
		(70, 100)	0.574	1,742	
		(105, 150)	0.826	1,211	
		(140, 200)	1.155	866	
ms-FROST	✓	(35, 50)	0.481	2079	adp-TS-UF-4
		(70, 100)	0.984	1016	
		(105, 150)	1.458	689	
		(140, 200)	1.914	522	
	✗	(35, 50)	0.519	1928	
		(70, 100)	1.025	976	
		(105, 150)	1.498	667	
		(140, 200)	1.954	512	
FROST2+	✓	(35, 50)	0.300	3,333	TS-SUF-4
		(70, 100)	0.542	1,845	
		(105, 150)	0.789	1,267	
		(140, 200)	1.095	913	
	✗	(35, 50)	0.463	2,160	
		(70, 100)	0.714	1,401	
		(105, 150)	0.961	1,041	
		(140, 200)	1.245	803	